Learning to Theorize in Mental Terms

MSc Thesis *(Afstudeerscriptie)*

written by

Gidon Kaminer

under the supervision of **Dr Fausto Carcassi**, and submitted to the Examinations Board in partial fulfilment of the requirements for the degree of

MSc in Logic

at the *Universiteit van Amsterdam.*

Date of the public defence:	**Members of the Thesis Committee:**
August 28th, 2026	Dr Fausto Carcassi (supervisor)
	Dr Malvin Gattinger (chair)
	Dr Martha Lewis
	Dr Giorgio Sbardolini

Institute for Logic, Language and Computation

Abstract

Contents

Chapter 1

Theory of Mind and its Acquisition

1.1 The nature and scope of theory of mind

the problem of other minds, scoping to propositional attitudes

The ice cream peddler's scratchy jingle got louder still as he brought his cart to a stop nearby. A child, hitherto engrossed in sandbox amusements, abandoned her playmates abruptly. She darted across the playground, hurtling at full speed towards a gaggle of parents. Absorbing the impact of a 20 kg cruise missile, the target continued his conversation unfazed by the girl now tugging at his jeans. Only when he deigned to look down at the child did he shriek and go pale. But the girl barely had time to react, because just then she heard her name shouted from the other side of the playground. She turned to its source and ran towards a similar-looking man whose outstretched arms held two ice cream cones.

You chuckle at the scene, observing from a nearby park bench. The girl, wanting ice cream, pestered a man she thought was her father. The man, undoubtedly accustomed to his own nagging kids, ignored the girl thinking it was his own child. Descriptions like these, in terms of desires, beliefs, and intentions, seem trivial and obvious. But there's something miraculous about them. How are you able to peer into other peoples' minds? Mental content is unobservable, all you see is a sequence of physical actions. And yet humans, even infants, are experts at inferring mental content rapidly and accurately.

Humans have the cognitive ability to impute mental states to others, to attribute a belief, desire, or intention to another person. This capacity is known as "theory of mind", "folk psychology", the ability to "mindread", to "mentalize". It is integral to everything that makes us human: to cooperate with someone you need to understand what they think, what they want. To decode someone's words, you need to begin with what they're trying to (by the Rational Speech Act framework). To understand social roles like "helping" and "hindering", you need to attribute an intention to them. Humans make these inferences so naturally that it seems like not much of an impressive ability at all, and "mindreading" seems too fabulous a term for something so mundane. But the ubiquity of these inferences should make the human capacity for mentalizing even more impressive, not less.

In this thesis we study the question: how is this capacity acquired? How do humans, even at a young age, come to acquire this ability? We'll begin by

in this section:

- **the problem of other minds** framed as a fundamental challenge: minds are unobservable, yet humans (even infants) are experts at inferring their contents rapidly and accurately

- **function of ToM** its role in social interaction, such as cooperation, communication (pragmatics, RSA), and understanding social roles like "helping" or "hindering"

"Theory of Mind (ToM) refers to the cognitive capacity to attribute mental states to self or others." Other names for the same capacity include:

- "commonsense psychology"
- "naive psychology"
- "folk psychology"
- "mindreading"
- "mentalizing"

[MSS12]

mental states include things like

- perception
- bodily feelings
- emotional states
- propositional attitudes (beliefs, desires, hopes, intentions)

[MSS12]

We'll be focusing on the attribution of propositional attitudes.

1.2 Experimental evidence: developmental milestones (ToM scale)

early roots (0-12 months)

discuss evidence that infants as young as 6 months old interpret motion as goal-directed and apply a "teleological stance", an intuitive schema relating actions, goals, and environmental constraints (this also helps later to defend `optimize`) as non-intensional

ToM scale: Diverse desires [Repacholi and Gopnik 1997 "Early reasoning about desires: evidence from 14 and 18 month olds"] 18 month olds can identify diverse desires and reason about them

Woodward 1998: attribution of object-directed goal in 5-6 month olds. Infants already see agentically

teleological stance [Gerg-ely, G., Nádasdy, Z., Csibra, G., & Biró, S. (1995). Taking the intentional stance at 12 months of age. Cognition, 56, 165–193]

epistemic access (12-24 months)

review findings on infants' sensitivity to perceptual access (e.g. knowing an agent can only have a goal for an object they can see)

ToM scale: Knowledge access

"Sally-Anne" False belief

milestone where children transition from reality-centric responses to representing false beliefs around age 4

ToM scale: False beliefs

Description of the false belief task [WP]. Results: 40% of the 4-year olds and 90% of the 6 to 7 year olds correctly predicted Maxi would look at the location where he believes the chocolate is, rather than its actual location. When information access or lack thereof was made explicitly salient, it rose to 50% of 4-year olds. But children under 3.5 had quite low rates of solving, and their performance didn't improve when the scenario was made more explicit. So the conclusion is that around 3 or 4 years old, children achieve the ability to attribute minds.

second-order mentalizing

brief overview of more complex social reasoning that emerges later (e.g. "I think that you think"), where the representations have to be richer, the model has to be higher-dimensional

1.3 What a good theory of ToM acquisition must explain

Before surveying various accounts of ToM acquisition, we specify some of the salient features of the ToM capacity that a good theory of ToM acquisition should account for. We derive these features from the experimental evidence in the developmental timeline presented in Section 1.2.

1.3.1 Mental content isn't in the data

Since a ball's position and velocity are recoverable from a sequence of images, a learner of intuitive physics can just extract regularities among the things they see. Learning intuitive psychology isn't like this. What is available to the observer is a trajectory of bodies through space, but what has to be acquired is a vocabulary of attitudes that don't appear visually in the trajectory. Mental states are underdetermined to the data. An agent walking across the room to the cupboard can be explained as them wanting the chocolate and believing it's there, or as wanting to confirm it has been moved and beliving it is there, etc. Any behavior is compatible with myriad belief-desire pairs, and no amount of observation will narrow the field. Yet humans select a fitting belief-desire explanation quickly, confidently, and on very little data.

The underdetermination problem presupposes a learner who already has the concepts and must choose among ascriptions. But a learner who lacks the concept *believes* can't even make an attribution. So ToM acquisition isn't just curve-fitting, parameter estimation within a given theory. Rather it's the construction of the theory's terms, the positing of unobservable entities to explain observable regularities. So an account of acquisition must explain how novel conceptual terms can come about.

1.3.2 The ToM capacity is structured and productive

Since "believes" can take any proposition as an argument, the set of available attributions is unbounded, i.e. ToM is a productive capacity. One can entertain attributions never previously encountered in the data, e.g. attributions of beliefs one takes to be false or about things that do not exist. And also the belief-attribution operator embeds naturally (it applies to its own output), which enables second-order mentalizing.

A good account should explain how it is that ToM is acquired in systematic families rather than independent items, e.g. that anyone who can represent that Sally believes the marble is in the basket must also be able to represent that Anne believes it is in the box [FP]. So ToM is inferentially coherent, in the sense that attributions are connected to one another by principles rather than being

connected to observations one at a time. The terms get their content from their role in a network of inferences rather than from links to observations. So an account of acquisition has to produce a representational medium in which content composes, embeds, and licenses inference.

1.3.3 ToM capacity arrives in stages

The milestones in Section 1.2 form an ordered sequence. Children reliably pass diverse desires before knowledge access, and knowledge access before false belief, and the ordering holds even for individual children, not just in group averages [WL04]. The timing of the false-belief transition in particular is one of the more robust findings in developmental psychology [WCW01].

An acquisition theory should explain the delay from being able to attribute goals (before 1 year old) to being able to predict false belief (around 3.5-4 years). It shouldn't just explain how the capacity appears, but also why it appears in this particular order, with later stages seemingly built off the backs of earlier ones. A good account should appeal to the cumulative nature of ToM acquisition in explaining it.

1.4 Theoretical perspectives on ToM acquisition

We'll now survey three approaches to ToM acquisition, which are instances of the broader cross-domain cognitive development debate. We'll evaluate each theory based on how well it accounts for the features of ToM outlined in Section 1.3.

1.4.1 Nativism and core knowledge

view that ToM is a built-in module, a core system [Leslie]. Discuss the "poverty of stimulus" argument, the data is too sparse for general learning. Which implies innate domain-specific constraints like the "principle of efficiency"

[Berwick 2011] for poverty of stimulus, and Chomsky 1971

"On one side, a purely nativist response was to deny learning and to focus on characterizing the detailed representations that were already in place" [GCT24]

Modularity-nativists argue for the existence of a Theory of Mind Module (ToMM), a natively-endowed domain-specific system for understanding the behavior of intentional agents. In Alan Leslie's characterization the ToMM introduces attitude concepts like *Believes* and uses a "proprietary representational system" for ascribing attitudes to agents [Les94]. Our goal is to demonstrate that the ability to theorize in mental terms can in principle be *learned*, without the need for a native ToMM.

THE DELAY: Against a strong nativism, the delay wants explaining. Goal attribution is in place well before the first birthday, and false-belief prediction is not reliable until around four. A modular account can appeal to maturation, or to performance demands masking an underlying competence [Les05, SL01], and this may well be right. But it means the developmental data is being accommodated by auxiliary hypotheses rather than predicted by the theory, which weakens the position considerably given that the developmental data is the main thing to be explained.

1.4.2 Empiricism and connectionism/associationism

view that ToM is learned through statistical patterns and bottom-up assocations from experience.

the structure exists, it's just not in the data. So how can a learner infer abstract concepts like "belief" if they aren't in the data?

[McClelland 2010] on emergence

[McClelland & Rumelhart, 1986] and [Rogers & McClelland, 2004] on connectionism

statistical models of learning defined over large numerical vectors [Ten+11], [GCT24]

"On another side, an empiricist response was to suggest that structured representations were not necessary, that learning (and the inferences that followed) was simply a bottom-up process of learning statistical associations" [GCT24]

THE DELAY: Against a purely associative empiricism, the order is not obviously the order of frequency in experience, and later competences appear to **presuppose** earlier ones rather than to be acquired independently alongside them. Belief attribution looks like something assembled out of goal attribution and perceptual access, not like a separate regularity learned from its own statistics.

1.4.3 rational constructivism and the theory theory

Introduce the "child as scientist" metaphor. children don't just associate, they construct causal models, intuitive/folk theories of the mind that they revise in light of new evidence, similar to scientific progress.

developmental (Piagetian) account of building up from smaller components

rational constructivism (reconstructing constructivism) adopts the constructivist idea of theory change but seeks a more formal mechanism than classic Piagetian accounts.

middle way between nativism and empiricism, where domain-general statistical mechanisms construct domain-specific structural knowledge

"how do our minds get so much from so little?" [GCT24]

how children's minds grow [Gopnik & Meltzoff, 1997]

Bayesian cognitive models explain how people learn with abstract structured knowledge [Gopnik & Tenenbaum, 2007; Griffiths, Chater, Kemp, Perfors, & Tenenbaum, 2010; Perfors, Tenenbaum, & Wonnacott, 2010; Griffiths, Sobel, Tenenbaum, & Gopnik, 2011b; Xu, 2019; Spelke, 2022]

unifying the two [GCT24], moving beyond dichotomies:

- empiricism versus nativism
- domain-general versus domain-specific
- logic versus probability
- symbols versus statistics

"How can domain-general mechanisms of learning and representation build domain-specific systems of knowledge? How can structured symbolic knowledge be acquired through statistical learning?" [Ten+11]

Powerful abstractions can be learned surprisingly quickly, together with or prior to learning the more concrete knowledge they constrain. Structured symbolic representations need not be rigid, static, hard-wired, or brittle. Embedded in a probabilistic framework, they can grow dynamically and robustly in response to the sparse, noisy data of experience. [Ten+11]

THE DELAY: This second point is the one this thesis takes most seriously, because it says that acquisition is **cumulative**: what is learned early becomes the material out of which what is learned later is composed. That is the bootstrapping picture [Car11, PTG12]. So the explanandum is not merely that the capacity appears, but that it appears in this order, with the later stages built from the earlier ones. A good account should make the ordering fall out of its mechanism rather than stipulate it.

A prior over hypotheses is a way of selecting among attributions that the data underdetermines; programs are a representational medium that composes, embeds, and executes; and library learning is a mechanism in which abstractions acquired early become the primitives from which later ones are built. Whether that mechanism actually delivers a mental vocabulary, rather than merely being able to express one, is the question the rest of the thesis puts to the test.

1.5 conclusion

The conclusion will be that each position captures one or two, that the theory theory comes closest, and that it does so by describing what happens rather than by naming a mechanism that would make it happen.

The theory theory scores best on the theoretical requirements stated in, but doesn't name a particular mechanism for how this works. Nativism's strength is parasitic on that absence.

I'm gonna end the section by noting that while the theory theory provides a powerful metaphor, it lacks a precise computational backbone to explain how children search through the infinite space of possible theories. Basically that the strength of the nativist argument lies in part in the supposed lack of algorithms for learning those structured representations. then this sets up chapter 2 to introduce bayesian program induction (via DreamCoder) as that formal implementation.

Chapter 2

Learning as Bayesian Program Synthesis

2.1 Levels of analysis and reverse engineering

Of the three positions surveyed in Section 1.4, the theory theory comes closest to accounting for the features of theory of mind laid out in Section 1.3: it takes seriously that mental terms are posits rather than observables, that they compose, and that they arrive in a cumulative order. But it accounts for these by describing what happens rather than by naming a process that would make it happen. The child constructs a theory, revises it in light of evidence, and builds later theories on the back of earlier ones. Which process does the constructing, and by what standard does it prefer one theory to another? Without an answer, the nativist can reply that no such process exists, and that what looks like construction is really maturation of something already in place. The strength of that reply is parasitic on the absence of a mechanism, and the business of this chapter is to supply one.

Supplying it requires first being clear about what kind of claim a mechanism is, and at what level of description it lives. This section fixes that vocabulary.

2.1.1 Reverse-engineering as a method

The approach taken here belongs to a tradition in cognitive science that Griffiths et al. call *reverse-engineering*: using the mathematical and computational tools developed in the engineering project of building artificial intelligence in order to understand the operation of human thought [GCT24]. Engineering, in the forward direction, starts from a computational problem and builds a system that solves it. Reverse-engineering runs the same relation backwards. We begin with a system — the mind — that manifestly solves some problem, and the first task is to say precisely what that problem is: what information goes in, what has to come out, and what would count as getting it right. Once the problem is stated in those abstract terms, we can ask which of the systems we know how to build solve problems of that shape, and then model the mind as a system of that kind.

This is not an argument that the mind is an artificial intelligence, and it does not licence reading the implementation details of an AI system back into the head. What it licences is weaker and more useful: if a problem has a certain abstract structure, then anything that solves it, biological or otherwise, is subject to the constraints that structure imposes. AI supplies a stream of candidate hypotheses about how a mind might work, and the correspondence between model behaviour and human behaviour is the evidence that adjudicates between them [GCT24].

2.1.2 Marr's three levels

The reason the order matters — problem first, mechanism second — is the argument Marr made for analysing information-processing systems at three distinct levels [Mar10]. In Tenenbaum et al.'s formulation [Ten+11]:

- the **computational level** "characterizes the problem that a cognitive system solves and the principles by which its solution can be computed from the available inputs in natural environments";
- the **algorithmic level** "describes the procedures executed to produce this solution and the representations or data structures over which the algorithms operate";
- the **implementation level** "specifies how these algorithms and data structures are instantiated in the circuits of a brain or machine".

The levels are related but not reducible to one another. The same computational problem admits many algorithms, and the same algorithm admits many physical realisations, so a description at one level underdetermines the levels below it. This is why reverse-engineering is a top-down, or function-first, strategy: one tries to understand what a system is for before diving into how it does it [GCT24]. The point is made explicit in Anderson's framework of *rational analysis*, which analyses cognition in terms of adaptive solutions to problems posed by the environment [Anderson 1990], and it is the level at which Bayesian models of cognition characteristically operate. The Bayesian framework begins with the logic of the inference made when generalising from examples, rather than with the algorithmic steps or the cognitive processes that carry it out [PTR11].

2.1.3 The computational problem posed by other minds

So we should ask first what problem a mindreader is solving. Section 1.3 has already supplied most of the answer. The data available to the learner is a trajectory of bodies through space. What must be recovered is the process that generated that trajectory, and that process runs through states — beliefs, desires, intentions — that never appear in the data. The trajectory underdetermines the process: any behaviour is compatible with indefinitely many belief-desire pairs, and no amount of further observation eliminates all but one.

Stated that way, the problem is an instance of a familiar one. Given data, entertain hypotheses about the process that generated it, assign each a prior plausibility, and revise those plausibilities as the data comes in. And there is a known answer to the question of how such revision *ought* to go: Bayes' rule specifies the ideal solution, and thereby generates predictions that can be checked against what people actually do [GCT24]. If human judgements line up with those predictions, we have grounds for thinking that this is the problem being solved — and a model we can use to build machines that solve it the same way.

That is the shape of the claim this thesis makes. The computational-level thesis is that acquiring a theory of mind is the problem of finding the best explanation of observed behaviour, where "best" is fixed not by taste but by the logic of Bayesian inference: a trade-off between the prior plausibility of an explanation and how well it fits the data. Section 2.2 makes that trade-off precise, and Section 2.3 argues that the hypotheses being traded off are best understood as *programs*.

2.1.4 What is claimed, and what is not

Fixing the level fixes the scope of the claim, and it is worth being explicit about what falls outside it. We are not studying how a child in fact comes to learn theory of mind. That is the province of developmental and cognitive psychology, and nothing here should be read as a proposal about neural implementation or about the moment-to-moment course of a particular child's development.

Nor is the claim that children run the algorithm presented in Section 2.6. ECD, the wake-sleep library-learning system described there and implemented in Chapter 4, is offered as an algorithmic-level *realisation* of the computational-level claim: a demonstration that the problem, so stated, is solvable by a mechanism that begins with no mental vocabulary at all. Its role in the argument is one of sufficiency rather than fidelity. If a learner equipped only with domain-general primitives and a preference for short programs comes to posit belief-like structure, then positing such structure is something a learner *can* do rather than something it must be given — and the nativist's inference from "no mechanism has been named" to "no mechanism exists" is blocked. Whether human learners use this mechanism, or some other one that solves the same problem, is a further question that this thesis does not settle.

There is a caveat to the tidy division of levels, and it matters for Section 2.5. Many early Bayesian models addressed the computational level alone, characterising cognition as approximately optimal statistical inference in a fixed environment without reference to how the computation is carried out. The hierarchical models discussed later sit somewhere between the computational and the algorithmic: they describe cognition as approximately optimal inference in probabilistic models defined over a learner's subjective and dynamically growing mental representations of the world's structure, rather than over some objective and fixed world statistics [Ten+11]. Once the hypothesis space is itself something the learner builds and rebuilds, the question of what the learner is doing can no longer be cleanly separated from the question of what it is doing it *with*. That entanglement is not a defect of the account; it is the part of the account that does the developmental work.

2.2 Bayesian inference: the logic of learning

explain prior/likelihood, size principle, Bayes as normative solution

2.3 Programs as the hypothesis space

- Language of Thought
- compositionality/productivity
- lambda calculus
- Fodor/Goodman/Rule
- symbols vs. vectors

bayesian inference over structured representations

contrast the symbolic approach (structured, compositional) with the connectionist approach (unstructured, associative weights), arguing that programs capture the productivity and open-endedness of human thought

concepts can be embedded and composed etc.

defining probabilities over structured symbolic forms of representation like graphs, grammars, predicate logic, relational schemas, and functional programs [GCT24]

only programs capture full breadth and depth of people's complex capabilities to udnerstand and execute algorithms[Goodman et al 2015]

with code we can model both procedural and declarative knowledge using a single format of representation [Rul]

Lambda calculus as formal language for compositional semantics [Heim & Kratzer, 1998] [Steedman, 2000]

and for other learning tasks [Piantadosi, Goodman, Ellis, & Tenenbaum, 2008] [Liang, Jordan, & Klein, 2009, 2010] [Zettlemoyer & Collins, 2005, 2007]

why programs for theory of mind?

By the classical theory of concept formation, concepts have a lot of qualities that are reminiscent of programs [Bru+09]

- concepts are represented compositionally
- concepts are logical combinations of features of objects
- concepts are rules for classifying objects
- concept learning is deducing correct classification rule

necessity of structured symbolic mental representations [Fodor 1975]

[Lake 2017]

how else would we capture/account for human's algorithmic abilities, if not with mental programs? [Goodman 2015]

- knowledge decomposes into concepts

mind is a program that corresponds to the world. and in fact that's how we can learn so quickly, [Baum 2003]

programs are the universal mental thing that can be instantiated in different mediums, languages, etc [Lupyan & Bergens 2016]

social reasoning requires recursive, compositional representations to handle nested mental states. There's no cleaner way of doing this than with programs. it's basically already a program.

learning is programming. learning a concept is building a program composed of lower level primitives, building an algorithm that does something. symbolic programs are the best formal knowledge representation [Rul]

embedded queries for choices of another agent, intuitive psychology [Stuhlmüller & Goodman, 2013] [Goodman 2015]

Human thinking as computation, computational model [Newell et al 1958, Newell et al 1959]

Thinking is a production system [Lovett & Anderson 2005]

symbolic programs give learner the freedom to adopt any syntax that is useful [Gopnick & Wellman 2012]

Siskind gives the example `lift(x,y)=CAUSE(x,GO(y,UP))` for knowledge as compositional programs [Siskind 1996]

2.4 Bayesian program synthesis

- Bayes over programs
- prior as program size/description length

Prior vs. Likelihood: learner trades off the simplicity of a program (the prior) against its fit to observed behavior (the likelihood)

Piantadosi 2012:

- likelihopod function that uses the size principle, penalize overly broad hypotheses [Tenenbaum 1999]
  - used for cross situational word learning [Frank, Goodman, and Tenenbaum (2007)]
  - used for solving subset problem in compositional semantics [Piantadosi et al. (2008)]
- prior is from rational rules model [Tenenbaum, Feldman, and Griffiths (2008)] which first linked probabilistic inference with formal, compositional, representations
  - assumes learners prefer simplicity

likelihood is: data have zero probability if they disagree with classificaiton rule, and constant probability otherwise [Goodman 2008]

Bayesian inference: rational framework for updating beliefs given observed data [Jaynes, 2003; Mackay, 2003]

background knowledge is constrained hyptohesis space, finer-grained knowledge is the prior degrees of belief in the hypotheses [Griffiths 2024]

The likelihood reflects the fit between hypothe- sis and data, and the prior indicates the a priori plausibility of a hypothesis (which might decrease for hypotheses that have low frequency, are complicated, or seem otherwise improbable). The contribution of these two factors to the conclusions that we should draw is fairly natural and makes intuitive sense in a variety of contexts. Returning to an exam- ple introduced in chapter 1, if you see John coughing (your data d), you might consider three hypotheses about the cause of the cough: a cold (h 1), lung disease (h 2), or heartburn (h 3). You might rule out heartburn on the basis of fit, since it might only slightly increase the chance of coughing. A cold and lung disease both fit well with the cough—they increase the probability of coughing—but they differ in plausibility. Normally, a cold is far more com- mon than lung disease, and thus might be the hypothesis that you would select to explain the coughing. However, the plausibility of these two hypotheses might change if you were passing by a hospital and saw John coughing inside. All inductive inferences require con- sidering fit and plausibility. Bayes' rule just tells you how they should be combined to reach a conclusion, using the common language of probability theory to determine the impact of each factor. In the context of cognitive science, priors become a useful way of describing the inductive [Griffiths 2004]

Without the constraints imposed by the prior, no meaningful generalizations would be possible. Without the likelihood, nothing could be learned from multiple examples beyond simply eliminating inconsistent hypotheses. … The prior determines which concepts count as "natural," whereas the

likelihood generates the specificity preference and determines how the strength of that preference—and thus the sharpness of generalization—increases as a function of the number of examples. [Xu 2007]

since knowledge is programs, learning is program induction [Kitzelmann, 2009; Flener & Schmid, 2008; Gulwani et al., 2017]

2.5 Learning the language itself

- HBMs
- overhypotheses
- blessing

2.5.1 Hierarchical Bayesian models

Blessing of abstraction, overhypotheses, HBMs

Explain how learning overhypotheses enables a learner to acquire the form of a theory (i.e. agents are rational) before learning specific goals

abstract knowledge can be acquired faster than specific facts, by pooling evidence across multiple scenarios. This could explain how children acquire core social constraints early in life.

theory of mind is the overhypothesis that structures instances

early Bayesian models addressed only the computational level, just stated what the system can do (optimal statistical inference), without specifying how it does it. HBMs get closer to the algorithmic level, showing "cognition as approximately optimal inference in probabilistic models defined over a learner's subjective and dynamically growing mental representations of the world's structure, rather than some objective and fixed world statistics." [Ten+11]

e.g. concept learning is set membership and causal reasoning is directed graph relations. but it's computationally prohibitive to list all the possible hypotheses, their priors and likelihoods, it's combinatorial. So there's higher level constraints like a bipartite graph or something (the mediacl example) [Ten+11]

General reference for HBMs, assumptions at mulitple levels of abstraction [Gelman, Carlin, Stern, & Ru-bin, 1995]

ideal learner of abstract knowledge is one that learns over a hierarchy of models [Tenenbaum, Griffiths, & Kemp, 2006]

The blessing of abstraction: Learning high level abstractions can be faster than learning low-level instances [KPT07]

HBMs for domain-specific abstract causal knowledge [KPT07]

HBMs for simple relational theories [KT08]

a learner who is simultaneously learning abstract and specific knowledge is almost as efficient as a learner with an innate (i.e. fixed) and correct abstract theory [GUT11]

HBMs for domain general causality learning [GUT11]

"hypothesis spaces of hypothesis spaces, with priors on priors" [GCT24] So over a longer timescale you can learn the priors needed for a specific learning task.

The overhypotheses constrain the space of lower-level hypotheses

[KT08] gives an example of how HBMs can be used to learn overhypotheses about feature-variability (e.g. the shape bias) which help to do categorization (the lower-level learning task)

[Ten+11] gives an example of HBMs for constraining directed graph models, which are then used for explaining observed events. First learn intervention-based causality, then use that to constrain inferences about instances of causality.

introduce the idea that learning is bootstrapped and "gets off the ground" by combining strong, potentially innate, domain-general mechanisms with minimal, skeletal domain-specific knowledge

Minimal nativism: "strong but domain-general inference and representational resources are aided by weaker, domain-specific perceptual input analyzers" [GUT11]

first the most abstract domain knoweldge comes into place and then the specific knowledge [Wellman and Gelman 1998]

[GUT11]

blessing of abstraction also defined in [PTR11]

2.6 The DreamCoder algorithm

2.6.1 Wake: exploration/enumeration/search (Bayesian inference)

The wake phase solves the inverse problem: searching for the program (intention) that most likely generated the data (actions)

[Ellis 2020] [Palmarini 2024]

2.6.2 Sleep: Abstraction (library learning, compression)

Explain how refactoring and compression automate the discovery of new social primitives, implementing the Theory Theory by allowing the learner to grow their own language of thought

[Palmarini 2024] [Ellis 2020]

2.6.3 Sleep: dreaming (amortized inference)

explain the neural recognition network as a way to "compile" slow, deliberative Bayesian search into fast, "intuitive" social reasoning.

Chapter 3

Bayesian Theory of Mind

3.1 Generative model of an agent

- MDP/POMDP
- utility
- principle of rationality
- intentional stance formalized

agents as approximately rational actors, inverse planning

defining the generative model. agents move in an environment (MDP/POMDP) to maximize utility efficiency

Bayesian inverse planning [Baker, Saxe, and Tenenbaum (2009)]

"naive utility calculus" [Jara-Ettinger, Gweon, Tenenbaum, and Schulz (2015)]

Assume that the observer sees the agent as a rational planner. Then planning computation is solution to markov decision process.

- input: utility and belief functions defined over agent's state space and state-action transition functions
- output: series of actions the agent should perfoirm to maximize utility or fiulfill goals [Lak+16]

then the observer can simualte what the agent would do if the agent had certain goals/policies etc, and thus infer those from what actually happens by comparing the simulation to reality. [Lak+16] analogy to physics simulations. Say I want to figure out

First we have principle of rationality, expetation that agents will plan rationally given their world model. Then we reverseengineer what their mental states are that caused their behavior, based onthat principle. Formalizing Dennet's intentional stance. [BST09]

Explanation by rationalization, Bayesian theory of mind framework [Baker 2012]

model causal relation betwene beliefs, goals, actions as rational probabilistic planning in markov decision problem. [baker 2012]

observer uses Bayesian induction to work backward from a trajectory to the most likely goals and beliefs

humans can easily observe agents' actions and then infer the beliefs desires and intentions that led to those actions, assuming that the agent is approximately rational and goal directed

[Baker, Saxe, and Tenenbaum (2009)], [Lucas, Griffiths, et al. (2014)], [Jern, Lucas, and Kemp (2017)], [Jara-Ettinger, Gweon, Schulz, and Tenenbaum (2016)] [Baker, Jara-Ettinger, Saxe, and Tenenbaum (2017)] [Ullman et al. (2009)]

Forward-planning/rational decisionmaking: given an agent's intentions etc, predict their future actions

inverse planning: given an agent's actions, what were their intentions/beliefs/world mdoel?

use Bayes rule to produce posterior over possibel mental states

Baker et al 2017

3.2 Inverse planning and the naive utility calculus

- Baker 2009/2012/2017
- Jara-Ettinger
- inverse RL

3.3 belief attribution and false belief in BToM

3.4 What these models don't do

Bayesian Theory of Mind *is* a theory of mind, which is handed to the model. The agent/goal/ belief structure, including the separation of believed world from actual world, is stipulated by the modeler. So BToM explains mature performance, not acquisition. A nativist can read it as evidence *for* innateness.

3.5 In this thesis

we run the bayesian engine detailed in chapter 2 on ToM tasks, and ask whether the structure ch3 stipulates is instead discoverable by compression.

Chapter 4.

The Model

4.1. The Learning Problem

In our model the learner is an observer, in a setup analogous to a child in a developmental psychology experiment. In each task the learner is shown a set of *scenes*, visual episodes unfolding oer discrete time steps. For each set of scenes, the learner's task is to recover the underlying process that explains the scenes, by positing canididate hypotheses until it finds the correct one. We model hypotheses as programs, so a hypothesis is correct iff executing the program reproduces the scenes exactly (every frame, not just the final one).

The learner's

Scenes come in families that

each emphasize a particular concept which explains the underlying process. In particular we study mental families, i.e. scenes that cannot easily be explained by extensional programs that just describe the observable surface of the scene. For such scenes the shortest available explanation must instead posit a representation held by an agent that diverging from the world, must attribute an intensional representation to an agent.

In our model, theorizing about the world is a program synthesis problem while concept learning is a library learning problem.

We ask whether a learner equipped with nothing mental to begin with will converge on a mental explanation because it is the most compressive one.

To learn, in our setting, is to grow one's conceptual vocabulary by enriching it with useful conceptual abstractions. So our claim is that a reusable conceptual abstraction corresponding to belief attribution appears in a library that began without one.

Section 4.2 through Section 4.5 fix what the learner is given, what it is shown, what it can express, and where it starts; Section 4.6 gives the inference procedure; Section 4.7 states what would count as the claim succeeding or failing.

4.2. What is given, and what must be learned

We argue that

The argument of Section 4.1 is only as strong as the audit of the learner's starting point. If any part of the apparatus — the interpreter, the type system, the primitive library — already encodes the world/model distinction, or already carries a notion of agency, then a discovered belief abstraction shows nothing, because belief was present from the start in a thin disguise. This section makes the audit explicit.

TODO: the given/not-given table. Left column, *given*: the minimal interpreter (Section 4.2.1); the domain-general core primitives (Section 4.5); the monomorphic type system; the MDL objective; the exact-match success criterion. Right column, *not given*: any primitive or type denoting an agent, a goal, a belief, or a mental state; the world/model channel distinction; any commitment in interpreter state to more than one representation of the world; any task-specific bias toward the belief families. Each row should point at the section where the corresponding claim is discharged.

4.2.1. The interpreter

A hypothesis is a transition function, not a scene. A trajectory scene is generated from its first frame by iterated application, so for a candidate transition function f and first frame x_0 the generated scene is fixed by the recurrence $x_{i+1} = f(x_i)$. That recurrence lives in the interpreter rather than in program space, and the choice matters for two reasons.

The first is compositionality. "Iteratively apply f, starting from x_0, for n timesteps" is structure common to every solution in the corpus. Were it part of the program, it would be baked into every abstraction the learner discovers: a corpus of ten falling-object scenes would yield not the reusable `λ0. (step down #0)` but an overspecified *apply `(step down #0)` from* x_0 *for `#1` steps*, whose output type is an entire rendered scene. Such an abstraction cannot be combined with another, so it could never serve as a component of the deeper compounds that belief requires, and library learning would be defeated at the first round. Keeping the recurrence out of program space keeps abstractions `fn`-typed and therefore composable.

The second is that the recurrence carries no information the learner could be credited with discovering. Since it is shared by every solution, what search is actually looking for is only f; the rest is what is needed to render a scene from f in order to check it against the observation. So we move the rendering machinery out of program space and into the evaluation framework (the interpreter is given in full in [app-interpreter]).

The interpreter is minimal by design, and its minimality is the load-bearing part of the audit. Its state at any step is a *single grid*. It does not encode a (world, model) grid pair, and it holds no registry of agents, goals, or entities: there is nothing in it that distinguishes a cell playing the role of agent from a cell playing the role of obstacle. There is likewise nothing representational about it, since it only ever builds up one sequence of grids. The ability to hold multiple representations of the world, to hold a representation that deviates from the world and to attribute that representation to a particular agent, is nowhere in the interpreter. So assuming the interpreter as innate to the system does not smuggle in a theory of mind. Whether a program opens a second, private channel is a *structural* property of the synthesized program — a matter of which types its nodes carry (Section 4.4.3) — not a commitment built into interpreter state.

Template tasks (Section 4.3) are evaluated against a second interpreter, which threads a working grid while pairing each frame with a *constant external* template and applying a commit policy of type `fn_p_g`. The distinction from the trajectory case is not incidental bookkeeping but a control on what the pair types can be taken to mean: here the second channel is a given input rather than a privately derived model, so a program that consumes the pair with `sync_to_world` is performing registration, not holding a belief. The same primitive vocabulary therefore does mental work in one

interpreter and non-mental work in the other, which is what lets the corpus price belief against genuinely non-mental competitors that use the very same combinators.

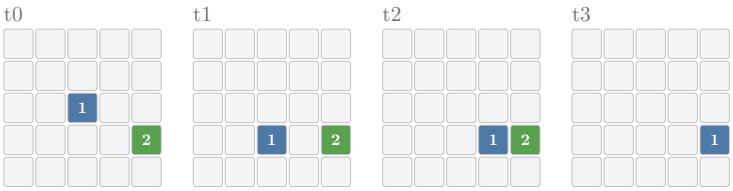
4.3. Scenes and the task corpus

We were inspired by experiments in the developmental psychology literature in which subjects are tasked with explaining a scene or making predictions about it. We created a corpus consisting of various grid-based tasks that depict some dynamic at play. The system is an observer trying to make sense of the depicted scene by finding a program that correctly generates the scene.

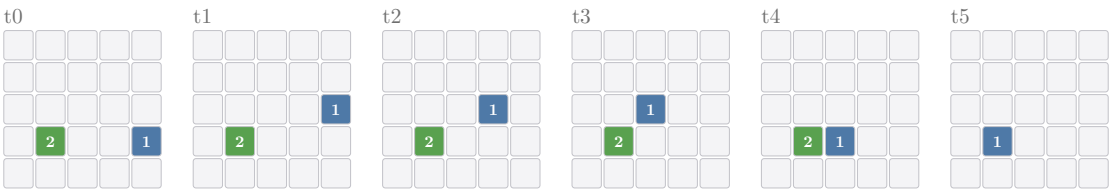
Most tasks in the corpus are *trajectory tasks*, a matrix A of dimensions $s \times s \times n$ that depicts a visual scene occurring over n time steps. Each frame is a *grid* of width and height s (we denote the i th grid as t_i). The solver just receives the initial grid t_0, the first frame (an $s \times s$ grid) of task A. Its goal is to find a program that generates the full A matrix, i.e. a program that posits the underlying data generation process. A correct solution is a function f that, given a grid t_i where $i < n$, returns the next grid in the sequence $f(t_i) = t_{i+1}$.

Tasks are classified by task type, by the general concept that explains the task. For example, the below is an instance of a *movement* task, which depicts a cell value (4) moving up steadily in some direction until it reaches an edge. A solution to the task might be program that encodes "at each timestep, value 4 moves one step up".

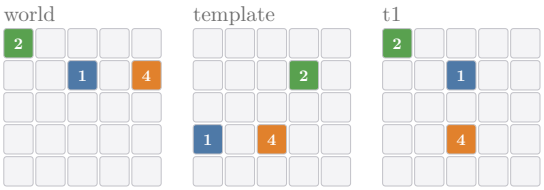
Another type of trajectory task is the *desire* tasks such as the instance seen below which depicts the value 1 getting closer to value 2 with each step. A solution to the instance might be a program that encodes "at each timestep, value 1 moves one step closer to value 2"



The above tasks (and most others in the corpus) can be solved by programs that explain the scene extensionally, a purely behaviorist visual description. But we're interested in studying tasks that depict agent behavior guided by a belief, tasks that can only be efficiently solved by attributing a belief to an agent. The below task is an instance of a *false wall* task,



Whereas a trajectory task's input is one initial grid t_0, a *template task*'s input is a pair of grids: a *world* grid and a *template* grid, both of dimensions $s \times s$. Given (*working*, *template*), the task is to produce t_1, a version of the working grid modified according to the pattern given in the template grid. This format is loosely inspired by the ARC-AGI-1 task corpus [CITE].



4.3.1. Tasks as sets of scenes

TODO: the k-scene task format. A task is not a single scene but k scenes (currently $k = 4$) sharing one latent program, and a candidate solves the task only if it reproduces all k. State why this is a model-level commitment rather than an implementation choice: a single scene underdetermines its generating program, so a program can reproduce one belief scene by coincidence — fitting the particular geometry of that scene rather than the dynamic it depicts. Requiring one program across k independently sampled scenes of the same family is what makes the identifiability condition of Section 4.3.3 bite, and it is what the rival measurements of Section 4.7 are measured against. Give the measured effect: rival programs that pass on one scene of the goal-displacement family fail on all four.

4.3.2. The families

Our corpus consists of various kinds of tasks. Most tasks can be solved without belief attribution, but some tasks require belief attribution.

require belief attribution:

- wall false belief: goal-oriented motion, navigating based on a believed grid which has a wall where one doesn't exist
- witness false belief: two agents, one has a false belief that a wall is there and navigates to avoid it, the other has a true belief about the world so doesn't avoid the wall
- goal-displacement false belief: classic sally-anne,
- false-obstacle belief
- dual belief

don't require belief attribution:

- simple movement
- desire: goal-oriented motion (uses `optimize`)
- overlay: moving object leaves a trail (uses `fork`)
- comet: goal-oriented motion with moving trail (uses `fork`)
- flee: like desire but with maximizing distance
- deletion: grid edit remove one cell
- denoise: grid edit remove a value
- underlay: complement to overlay

- obstacle: goal seeker detours. the grid doesn't have an obstacle there, but the goal seeker detours nonetheless. So the best explanation is `compose( wall_at r c, optimize(neg_dist gv) av )`, that the agent actually is navigating by a real wall that is there, and it's just that I the viewer don't see it
- relocation: move a wall from one location to another
- registration: snap one object onto the template
- perception: registration complement, record a sensation into the private channel
- multi-registration: snap all objects
- registration-except:

4.3.3. Identifiability of the mental families

The corpus only tests what it claims to test if the belief families are not solvable extensionally. A scene in which an agent detours around a wall that isn't there admits a mental explanation, but it may equally admit a shorter non-mental one, and if it does then a learner that solves it has demonstrated nothing about mentalizing. So membership in a mental family is a condition on the scene, not a label attached to it: the scene must be one that no program describing only its observable surface can reproduce.

The condition can fail in ways that are easy to miss. A wall false belief scene might place the falsely-believed wall at a location nowhere near the agent's path to its goal, in which case the agent navigates to the goal exactly as it would have without the false belief, and positing the belief is neither necessary nor helpful in explaining what is seen. Such a scene is underdetermined and is excluded from the corpus. The generators enforce the condition by rejection, and the checks they apply are given in [app-generators].

4.4. The hypothesis space

4.4.1. Programs as typed expression trees

A hypothesis is an expression tree over the library. Each node carries a type, together with the types of the arguments it expects; a node with no arguments is a terminal whose value is itself, like `up :: dir` or the literal `4 :: cellvalue`, and a node with arguments is an operator, like `step :: cellvalue, dir -> fn`, which is applied to its filled-in arguments. The tree is directly executable, and evaluating it bottom-up yields the object its root type names — so `(step 4 up)` evaluates its two leaves to the cell value `4` and the direction `(-1,0)` and applies `step` to obtain a transition function `fn :: grid -> grid`.

Enumeration is type-directed: an argument slot draws only from symbols whose type matches, so no ill-typed tree is ever built. This is why the type system is a substantive part of the model rather than bookkeeping — it is the type discipline that determines which compounds are even expressible, and therefore what the learner could in principle discover.

An invented abstraction is a node like any other, but it names a body: a tree over lower-level primitives with numbered holes standing in for its parameters, which is substituted in when the node is called. The point of naming is that the name is what gets charged. A program's description length is its node count, one for a leaf and one plus its arguments for an operator, and an abstraction used as a token adds just one, because the subtree it abbreviates lives inside the body and does not

appear in the program that uses it. Description length is thus the quantity in which library learning trades: an abstraction converts many charged nodes into one. The representation that implements this is given in [app-delta].

4.4.2. The library

The library is the learner's language of thought, and it has two parts: the `core` primitives a run begins with, and the `invented` abstractions that compression adds to it as they are discovered. Search always draws from their concatenation, so an abstraction discovered in round N is available as an atom in round $N + 1$ on exactly the same footing as a primitive that was given.

The library also fixes the prior. Symbols are grouped by return type, and a symbol of type τ is priced at $-\ln$ of the number of symbols sharing τ — the type-uniform distribution. So the grouping does double duty: it gives the branching factor at each argument slot and the cost of each choice at that slot (Section 4.6.1). It also means growing the library is not free, which is what makes the compression tradeoff of Section 4.6.2 non-trivial.

4.4.3. A monomorphic type system

Types are plain strings. A 2d array, i.e. one frame of a task, is of type `grid`, so a transition function is `fn :: grid -> grid`. A z-stack of grids, i.e. a sequence of frames (a full task), is of type `mat`. Cell values, e.g. a value representing an entity on the grid like a `4` moving up each timestep, are of type `cellvalue`. Grid positions are of type `coord`. These two int terminals are split by type so that diversifying cell values does not inflate the branching at a position slot (Section 4.5).

The type system is deliberately monomorphic: there is no polymorphism, so every arrow over a different shape of data needs its own named type and its own typed composer. This is a cost — a single polymorphic `compose : (b -> c) -> (a -> b) -> (a -> c)` would subsume many of the types below — but it is a principled one: keeping the primitives ground (rather than schematic) is what lets belief emerge as a discovered *composition* of concrete operations rather than being smuggled in as a polymorphic combinator (Section 4.5.3). The types beyond `grid`/`fn` are:

- `pair_gg` — a pair of grids `(grid, grid)`, the (world, model) channel pair;
- `fn_g_p` — a pair producer `grid -> pair_gg` (e.g. `dup`);
- `fn_p_g` — a pair consumer `pair_gg -> grid` (e.g. `sync_to_world`, `overlay`);
- `fn_p_p` — a pair endomorphism `pair_gg -> pair_gg` (e.g. `mapsnd`);
- `fn_g_c` — a coordinate reader `grid -> coord` (e.g. `locate`);
- `fn_gc_g` — a coordinate placer `(grid, coord) -> grid` (e.g. `place`);

and, for the arity-generalization phase, the recursive grid-stack `gstack` with its lift/endomorphism/render arrows `fn_g_s`, `fn_s_s`, `fn_s_g` (Section 4.5.4). Because whether a program opens a private second channel is now a matter of which of these types its nodes carry, the presence of a (world, model) representation is a *structural* property of a synthesized program, not a commitment baked into the interpreter's state.

4.4.4. Types and task roots

Which type counts as a solution is fixed by the interpreter the task is enumerated against. A trajectory task's input is a single initial grid, of type `grid`, and its solution is a transition function of

type `fn`, since for a grid at timestep i it must return the grid at $i + 1$. So only an `fn`-typed program is a valid solution for a trajectory task: the interpreter `unfold` takes the initial grid (`grid`), a transition function (`fn`), and the duration (`int`), and returns a `mat` to compare against the task.

A template task's input is instead a pair `(grid, grid)` — a working grid and a constant external template — and its solution is a *commit policy* of type `fn_p_g`, answering "what should each frame do with the template?"; these tasks are enumerated against `unfold_with_template`. Enumeration is therefore organized by root type, `fn` and `fn_p_g` being searched as separate cost-ordered streams, while compression pools the solutions of both roots into a single stitch call (Section 4.6.2) — so an abstraction for belief must earn its place against the whole solved corpus, not just its own root's share of it.

4.5. The primitive library

The library a run begins with is the model's substantive claim about the learner's initial endowment, and the three phases below weaken that endowment in turn. Each phase hands the learner a strictly more elementary starting vocabulary, so that a belief abstraction discovered in a later phase is discovered from less.

4.5.1. Core primitives

every run's library starts with core primitives used to construct the transformation functions. `step :: cellvalue, dir -> fn` moves the value one step in a specified direction, chosen among `left :: dir`, `up :: dir`, etc. `optimize :: util, cellvalue -> fn` moves the value one greedy (BFS-optimal) step towards improving its utility, chosen among `distance :: cellvalue -> util` or `neg_dist :: cellvalue -> util`.

TODO: the full core primitive table (symbol, type, gloss), or a pointer to it in the appendix.

4.5.2. Atomic (phase 1)

In our first trial (phase 1) we show that, given the domain-general capacity to represent counterfactual world states, a system can learn a functional abstraction that encodes a notion of belief attribution. The learned function attributes a counterfactual world state to a particular agent, determines what the agent's movement would be based on its own private (non-world) representation, and then renders only the agent's movement as a real-world artifact (so it doesn't render the agent's believed world as the real world). So this use of the capacity for counterfactual representation is discovered in program space.

The capacity is given as two primitives:

- `fork :: fn -> (grid, grid) -> fn`
- `sync_to_world :: cellvalue -> (grid, grid) -> grid`

4.5.3. Decomposed (phase 2)

In the second trial (phase 2), we show that the system can arrive at an abstracted notion of belief starting from an even more basic language of 2-ary combinators. We decompose the counterfactual representation machinery, initializing the system with a library of product-category combinators that are unambiguously domain-general. We show that enumeration recovers the basic structure of belief attribution even from these atomized primitive operations.

The product-category combinator family in our type system is

```
dup_g       : grid -> pair_gg
mapsnd      : fn   -> fn_p_p
compose_gp  : fn_g_p, fn_p_p -> fn_g_p
pipe_gpg    : fn_g_p, fn_p_g -> fn
```

which are just the pair-grounded instantiation of universal categorical combinators

- `dup     : a -> pair a a` (the diagonal $\triangle$)
- `mapsnd  : (b -> c) -> pair a b -> pair a c` (the bifunctor 'second')
- `compose : (b -> c) -> (a -> b) -> (a -> c)`

The decomposition is polymorphic in principle. Our monomorphic primitives are the ground instances at `a = grid`. So belief *can* be assembled from standard categorical combinators.

so if we had polymorphism we could collapse the above family into one `compose`.

4.5.4. Arity (phase 3)

TODO: the arity-generalization phase. The `pair_gg` type of phases 1 and 2 stipulates that a learner holding a private representation holds *exactly one*, which is a commitment the model should not be making on the learner's behalf — a one-model frame is precisely what a theory of mind is claimed to have. Replacing the pair with the recursive grid-stack `gstack` (`cons`/`nil`, with the lift/endomorphism/render arrows `fn_g_s`, `fn_s_s`, `fn_s_g` of Section 4.4.3) makes channel arity a free parameter, so the number of private channels becomes something search selects rather than something the type system fixes. State the verdict: MDL never selects more than one private channel, so belief's single-model frame is discovered rather than stipulated.

4.6. Inference

ECD is a wake-sleep library-learning loop inspired by the DreamCoder [Ell+20]. Each round consists of an enumeration phase, a compression phase, and a dreaming phase, and the three correspond to the three moves of the previous chapter: search is Bayesian inference over programs under a prior; compression is revision of the language in which hypotheses are written; dreaming is amortization of the first by a learned recognition model.

4.6.1. Search

In principle the enumerator doesn't itself set out to synthesize a solution program for each particular task. Rather it generates candidate programs arbitrarily from the grammar in order from most to least probable (from lowest to highest cost), checking candidate programs against the task corpus.

Say the task to solve depicts the value 3 rising in each successive grid, so the target solution is the program `(step 4 up)`. Its head is `step :: cellvalue, direction -> fn` and its leaves are `4 :: cellvalue` and `up :: direction`. A program's cost is given by the sum of the costs of its constituent primitives. Say Q is the initial flat prior, the type-uniform distribution, so each primitive's cost is the negative natural log of the number of symbols in the grammar that share its type. For example, there are $N = 4$ symbols of type `direction` so each gets $p = \frac{1}{N} = \frac{1}{4}$. By Shannon, we have cost(`up`) $= -\ln p = -\ln \frac{1}{4} = \ln 4 \approx 1.386$ nats. By the same process, `step`'s cost is $\ln 8 \approx 2.1$ nats and `4`'s cost

is $\ln 10 \approx 2.3$. By logarithmic additivity, to find the program's total cost we simply sum the costs of its constituent primitives which yields ≈ 5.8 nats for `(step 4 up)`.

Search proceeds cheapest-first, so the solution it returns for a task is the shortest program that solves it under the current library — the maximum a posteriori hypothesis, given that the prior prices a program by its description length. A candidate counts as solving a task only if the interpreter, run on the task's own first frame, reproduces the task's every frame exactly. Each still-unsolved task is enumerated as its own cost-ordered stream, which is what allows the prior Q to be conditioned on the individual scene rather than shared across the corpus — a fact we rely on in Section 4.6.3. The mechanics of cost-bounded search are given in [app-enumeration].

4.6.2. Library revision

While Enumeration minimizes the cost of each program under a fixed grammar, Compression lets the system revise the grammar in light of the solutions found by enumeration. It identifies syntactic structure common across found solutions, and abstracts it as a new primitive to add to the library. The goal of compression is to minimize the joint cost of the library plus the programs written in it, given by:

$$\min_{\text{library}} \left(\text{DL}(\text{library}) + \sum_{\text{tasks}} \text{DL}(\text{program} \mid \text{library}) \right)$$

The choice of whether to abstract a given shared structure is non-trivial. Consider a program tree fragment that recurs across many solutions. Say it gets added to the library as a new primitive `fn_k`, so now each subsequent usage of the fragment collapses from a multi-node subtree to a single leaf, so it costs one token rather than the several that `fn_k` abbreviates. But the library is now larger, and since a symbol's cost is $-\ln$ of the number of symbols sharing its type, adding one more `fn_k`-typed symbol raises the cost of every primitive of that type. The abstraction is worthwile only when the fragment is large enough, and recurs often enough, that the tokens saved across the corpus outweigh the price of carrying one more symbol. So accidentally-shared structure is never abstracted, while shared structural motifs do.

Where DreamCoder [Ell+20] searches for abstractions bottom-up, by enumerating refactorings of each found program and intersecting them, we delegate the search to the top-down Stitch [Bow+23] library-learning system, which searches the space of abstractions directly and so never has to materialize every rewrite. The search procedure and its utility bound are given in [app-stitch].

enumeration is by root types so `fn` and `fn_p_g` are searched separately. but compression is joint over root types, their solutions are pooled into a single stitch call. so an abstraction for belief has to fight over the space of all solved programs, including those from the other root type. so it really is that much more efficient. This preempts the attack that by separating the search to different root types we're biasing abstraction creation towards the belief tasks since they make up a higher percentage of `fn`-rooted tasks rather than `fn_p_g`-rooted. the same objective that could have paid for belief's structure is free to spend its budget on the overlay, registration, and obstacle families instead, but it ends up going to the belief families because those abstractions compress so much structure.

What compression returns is a set of abstractions together with the corpus rewritten to use them. Each abstraction is registered as a new symbol in the library, and the rewritten corpus becomes both the training data for the dreaming phase and the starting point for the next round — in which programs a level deeper, belief compounds too expensive to enumerate from bare primitives, have become reachable because their shared core now costs a single token.

4.6.3. Amortization

Enumeration searches under a prior Q that is fixed across the whole corpus: every task is enumerated against the same type-uniform distribution. But the tasks are not interchangeable. A scene in which the agent takes a straight path to its goal and a scene in which it detours around a wall call for very different transition functions, and a prior that cannot see the scene has no way to tell them apart — it must spend the same budget windows on both.

In the dreaming phase we train a recognition model [Ell+20], a neural network that reads a task x and emits a task-conditional prior $Q(\cdot \mid x)$, biasing the grammar toward the primitives that the scene is likely to need. It is trained by the wake–sleep procedure of a Helmholtz machine on two sources of program–scene pairs: *replays*, the solutions enumeration has actually found, run back through the interpreter on their own task grids; and *fantasies*, programs sampled from the current library prior and executed on freshly generated grids, which supply new pairs at no labelling cost where replays are scarce. The architecture, its training regime, and the rescaling that puts its output on the enumerator's cost scale are given in [app-recognition].

Two of its commitments are part of the audit of Section 4.2 rather than matters of engineering. The first is that entity *roles are read off from motion, not from cell values*: the corpus deliberately uses diverse agent and goal identifiers, so the encoder cannot key on "value 1 is the agent", and instead labels a cell occupied in the first frame but vacated by the last a *mover* and a non-background cell occupied at both ends a stationary *goal*. Roles in fantasies are likewise decided by the sampled program's motion, so a fantasy only ever *lets* a program drive two movers; it never manufactures a belief scene by hand.

The second is that the dreamed prior is gated to families with at least one solved instance. A family with no solves is invisible to the recognition model and could only be mispriced by it: it would flood the early budget windows with the non-belief primitives the model *has* seen and thereby delay the primitives that family actually needs. Such families stay on the uniform baseline and earn the dreamed prior only once they are solved and can contribute replays of their own. So dreaming accelerates the families the curriculum has already reached, and never the frontier it has not — which matters here, because belief is the frontier: the first belief solve cannot be an artifact of a prior trained to expect belief.

4.7. What would count as success or failure

TODO: the discovery criteria and the rival criteria — the section that makes the next chapter's measurements tests rather than descriptions. It should fix, in advance:

- **What counts as having discovered belief.** Not "the belief tasks were solved" — a solved belief task with an extensional program is a failure of the corpus, not a success of the learner. The criterion is structural: an abstraction appears in the library whose body opens a private

channel, derives a world state that diverges from the actual one, evaluates the agent's action against that divergent state, and commits only the action back to the world. State it as a condition on the term, so it is checkable.

- **That it must be a compression win, not merely reachable.** The discovered compound has to be selected by the joint objective of Section 4.6.2 against the non-mental rivals available in the same library — this is what the MDL margin measures, and the margin must be reported against the shortest rival the library can express, including the witness agent.
- **That it must generalize behaviourally.** A learned belief term should predict, on a held-out scene, that the agent goes where it believes the goal to be rather than where the goal is. This is the false-belief test proper, and it is the criterion that distinguishes a compressive coincidence from a term that means what we say it means.
- **What would refute the hypothesis.** Any of: no belief-structured abstraction enters the library within budget; one enters but loses on description length to an extensional rival; a rival that never opens a private channel reproduces all k scenes of a mental family; the belief term fails the held-out behavioural probe. Say which of these the runs in fact rule out, and which remain open — the unsolved dual-belief family should be named here rather than left to the results chapter.

Chapter 5

Results and Discussion

belief/theory-of-mind is a discoverable compound under MDL rather than a built-in primitive

5.1 Found abstractions

5.1.1 atomic primitives

round 1 enumeration solves obstacle tasks

```
[  1037] caught (compose (optimize (neg_dist 5) 4) (wall_at c1 c0))
```

and simple a few (7) simple false belief tasks

```
[102603] caught (fork (compose (wall_at c1 c2) (optimize (neg_dist 1) 7)) (sync_to_world 7))
```

so at the end of the round it abstracts a function for placing a wall and navigating around it

```
fn_0: (compose (wall_at #3 #2) (optimize (neg_dist #1) #0))  [cellvalue, cellvalue, coord, coord] -> fn
```

and uses `fn_0` to abstract the belief signature `fork(derive, commit)`: on the model grid, run `fn_0` as the derive, then run `sync_to_world` as the commit using the same `#0` (agent value) as passed to `fn_0`. So run the calculation for `#0` based on the modified grid, and then commit the conclusion to the real world.

```
fn_3: (fork (fn_0 #0 #3 #2 #1) (sync_to_world #0))  [cellvalue, coord, coord, cellvalue] -> fn
```

round 2 enumeration solves the rest of the simple false belief tasks using `fn_3`, requiring just around 1% of the original enumeration cost. So it can successfully use the learned abstraction to generalize. It's much easier to solve the false belief tasks when it already has a function and just needs to fill in the values, rather than needing to rediscover the entire structure of belief again

```
[  1053] caught (fn_3 4 c3 c2 5)
```

at the end of the round it abstracts the belief signature again, which was `fn_3` in round 1

```
fn_6: (fork (compose (wall_at #3 #2) (optimize (neg_dist #1) #0)) (sync_to_world #0))  [cellvalue, cellvalue,
coord, coord] -> fn
```

and it abstracts simple desire

```
fn_7: (optimize (neg_dist #0))  [cellvalue, cellvalue] -> fn
```

so then in **round 3** it can use those to solve witness-belief tasks

```
[ 11634] caught (compose (fn_6 5 8 c3 c3) (fn_7 2 1))
```

final abstractions: belief constructor

```
fn_6  [cellvalue, cellvalue, coord, coord] -> fn
body: (fork (compose (wall_at $3 $2) (optimize (neg_dist $1) $0)) (sync_to_world $0))
```

5.1.2 decomposed primitives

final abstractions

```
fn_6  [cellvalue, cellvalue, coord, coord] -> fn
body: (pipe_gpg (compose_gp dup (mapsnd (compose (wall_at $3 $2) (optimize (neg_dist $1) $0)))) sync_all)
  *** AGENT TYPE CONSTRUCTOR (belief — degenerate scope-complement commit) ***

fn_7  [cellvalue, cellvalue] -> fn
body: (optimize (neg_dist $0) $1)
  (desire fragment)

fn_8  [cellvalue, cellvalue, coord, coord] -> fn
body: (compose (wall_at $3 $2) (optimize (neg_dist $1) $0))
  (obstacle/belief policy: stamp wall ▸ navigate — the shared derive)

fn_9  [coord, coord, cellvalue, cellvalue] -> fn
body: (compose (optimize (neg_dist $3) $2) (wall_at $1 $0))
  (obstacle/belief policy: stamp wall ▸ navigate — the shared derive)

fn_10  [coord, cellvalue, cellvalue] -> fn
body: (pipe_gpg (compose_gp dup (mapsnd (compose (wall_at c2 $0) (optimize (neg_dist $1) $2)))) sync_all)
  *** AGENT TYPE CONSTRUCTOR (belief — degenerate scope-complement commit) ***

fn_11  [fn, fn_p_g] -> fn
body: (pipe_gpg (compose_gp dup (mapsnd $0)) $1)
```

5.2 The structure of theory of mind

explain the structure of ToM, of the agency signature, of belief attribution

belief is a function that takes a proposition and an agent

```
fn_0 = (fork (compose (wall_at $3 $2) (optimize (neg_dist $1) $0))
             (sync_to_world $0))
```

in which the agent value `$0` appears twice. once in the policy that plans over the private, phantom-walled grid built by `derive`, and again in the `sync_to_world` commit that writes the result back to the world.

Nothing in the grammar forced those two argument slots to be filled by the same value. The fact that they collapse into one shared hole is the structural signature of agency, of an agent acting

on its own model of the world. This signature is discovered by the description-length objective, it's not stipulated by a primitive.

Dennet on intentionality: it's not a specific structure of "belief", rather just something that plays that role

5.3 Scaffolding

Piaget's theory of stages of cognitive development [CITE]

Wellman's theory of mind scale [CITE]

show that lower-level solutions to non-mental tasks are building blocks for mental task solutions

5.4 Benefits of concept learning

The guiding motivation of this project is that an intensional theory, attributing a belief to an agent, is the most compressed representation of an agent's behavior. This is obvious, since the agent navigates based on its intensional representation, so a theory that accounts for this will better represent the underlying data generating process. It's certainly possible to describe an agent's movement without attributing a belief to it, but it would be a laborious explanation that accounts for each step in the sequence. And of coruse it doesn't generalize well, it overfits to a specific agent's movement in a particular scene.

To illustrate this we show that no non-intensional solution is as short as one which stipulates belief attribution. As described in chapter 3 the enumerator processes candidate programs in $-\log p$ order and saves just the first (cheapest) solution it finds, call it *found*. For false belief tasks should be a program that attributes a belief to an agent. Since a false belief task can also be solved by a purely extensional program, call any such program a *rival* since it's an extensional rival to the intensional explanation.

For each rival, we generate a *margin* score by taking $\mathit{margin} = \mathrm{DL}(\mathit{rival}) - \mathrm{DL}(\mathit{found})$, where DL is the description length (how many nats it takes to write program p under library L). So for any rival, if $\mathit{margin} > 0$ then the intensional program is shorter than the extensional, otherwise an extensional program is at least as short. We plot the empirical cumulative distribution over the margin values for all rivals, so the x-axis shows possible margin values and the y-axis shows the cumulative fraction of rival programs with $\mathit{margin} \leq x$. Since description length depends on the given library we plot two ECDFs: dotted for programs written in the base library, and solid for programs written in the final library (enriched with abstractions found through five ECD rounds).

We first give the plot for the higher level DSL in which `fork` and `sync` are given as atomic primitives.

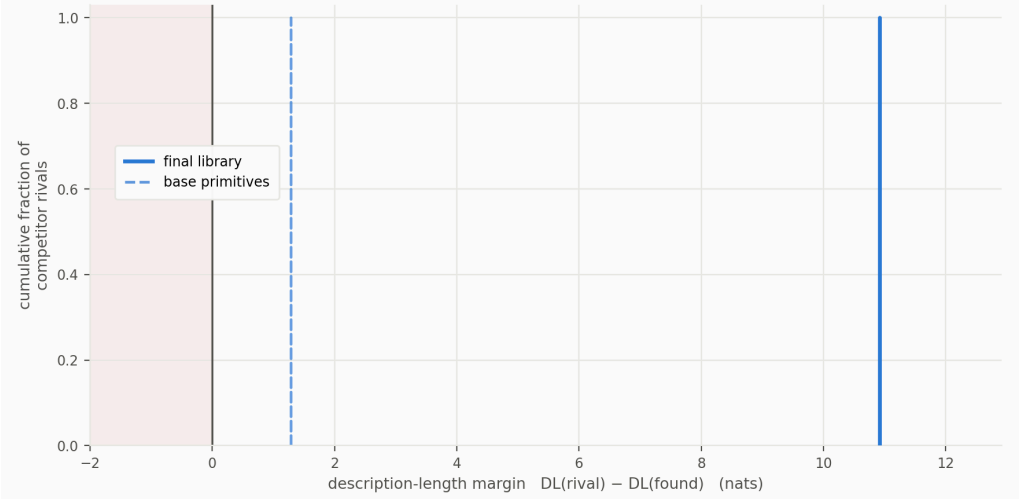


Figure 1: ECDF of MDL margin over rival programs (atomic DSL)

Both curves sit entirely to the right of $x = 0$, so for each of the 24 false belief tasks, the *found* (intensional) program is a shorter description length than any *rival* (extensional) program. In the base (dashed) library, rival programs cost only ≈ 1.3 nats more than the found program. But the margin baloons to ≈ 10.9 in the final (solid) library: once the `fork(derive, commit)` abstraction is added, the enumerator can immediately generate programs that posit an agent's intensional state. Without such a learned abstraction, an intensional program is only slightly less verbose than an extensional one. But with the abstraction, intensional programs are vastly more compressed than extensional ones.

This effect is even more pronounced in the plot for the lower-level DSL, in which `fork` and `sync` are decomposed to their product category components.

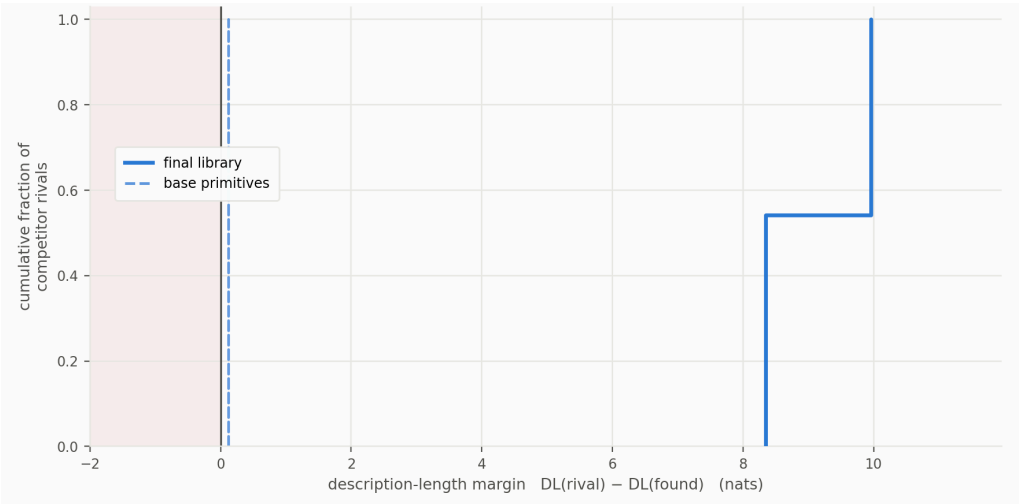


Figure 2: ECDF of MDL margin over rival programs (decomposed DSL)

Under the base primitive library, the mental reading is barely shorter than the physical one (≈ 0.1 nats). Since the intensional machinery of forking and syncing needs to be constructed from lower-level components, length of intensional programs end up virtually tied with that of the verbose extensional programs. But once the belief abstraction `fn_0` is learned and added to the library, the margin grows to $\approx +8.4$ nats. So mental programs go from being just as bad as non-mental ones to being vastly better, through learning a fitting abstraction. Note that the jump to a $\approx +9.96$ nats

margin for the last 11 tasks is due to a further specialized `fn_4 = (fn_0 ... c2)` that hard-codes a phantom-wall at a particular coordinate `c2` so such programs are one coordinate cheaper. But the rival cost is constant accross both, so the entire step is a property of how tightly the library compressed the intensional side.

We can also see this effect when comparing description lengths of different families of tasks: belief, desire, physics, and world. To plot, we fix each corpus (task type) and calculate each corpus's description length given a round as such:

$$\mathrm{DL}(C|r) = \sum_{c \in C} \mathrm{DL}(p_c|L_r) + \mathrm{DL}(L_r)$$

where C is a task corpus (e.g. belief tasks), c is an instance of a task in that corpus, r is an ECD round, and p_c is the found (minimal) program that solves task c. So calculate the description length of a corpus at a given round, for each task in the corpus $c \in C$ we sum the DL of each program p_c that solves it, given under that round's library L_r. And we add the description length of the library L_r itself, a structure term to account for the cost of a more complicated library.

Under the base library, total description length (black line) of the entire task corpus (every C) is 3209 nats. After four ECD rounds it drops to 2076 nats, an 1134 nat compression. We see the biggest drop in the belief corpus, once the system learns to efficiently represent intentional agents using the `fork(derive, commit)` abstraction. Once that constructor is in the library, every belief task rewrites through it and the corpus gets dramatically shorter at once.

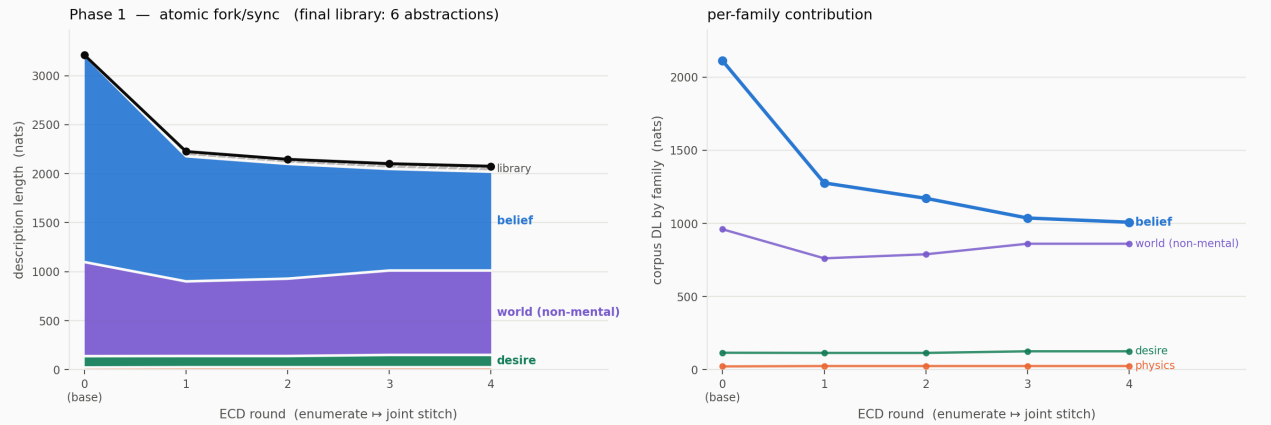


Figure 3: Description length per task family (atomic primitives)

Unlike the belief corpus, the description length of the other (non-mental) tasks remains mostly stable as new abstractions are learned. They're solved just as easily under the base primitives as they are under the final library (a bit worse actually since the library is more complex). If the initial primitives *did* encode mental notions, if there were a native ToMM, we'd see similar results for the belief tasks: they'd be solved just as easily from the start. But since iterated abstraction yields a significant improvement, it's clear that the initial primitives are not mental but that the ECD loop enriches the library with abstractions that *do* encode mental concepts (the concept of belief attribution).

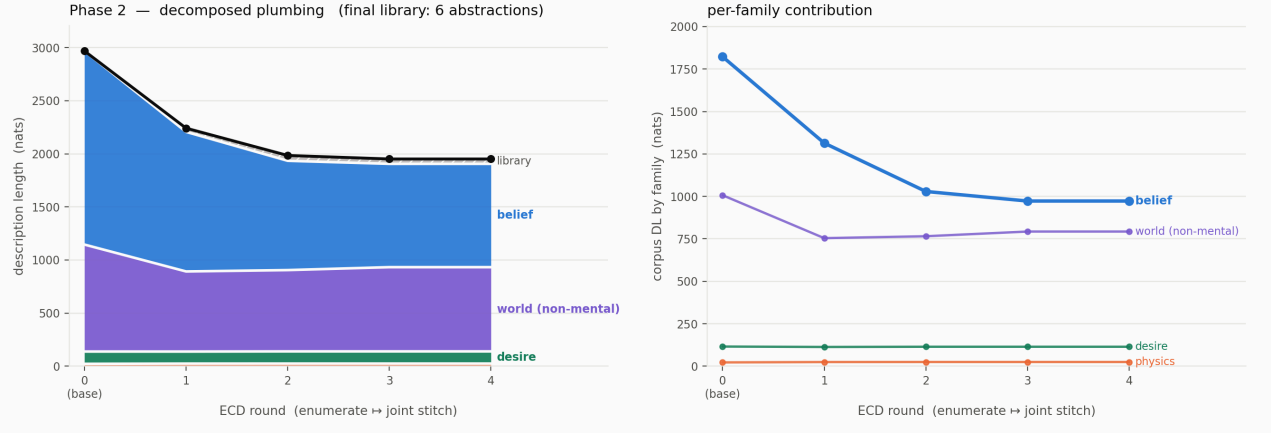


Figure 4: Description length per task family (decomposed primitives)

Belief tasks are reached late and cheaply. They only become solvable once the ECD loop has abstracted the reusable parts, and the moment that abstraction is added to the library, belief tasks flip from timing out to being solved in seconds. See the following plots of cumulative task solutions per round, and per-task solve time.

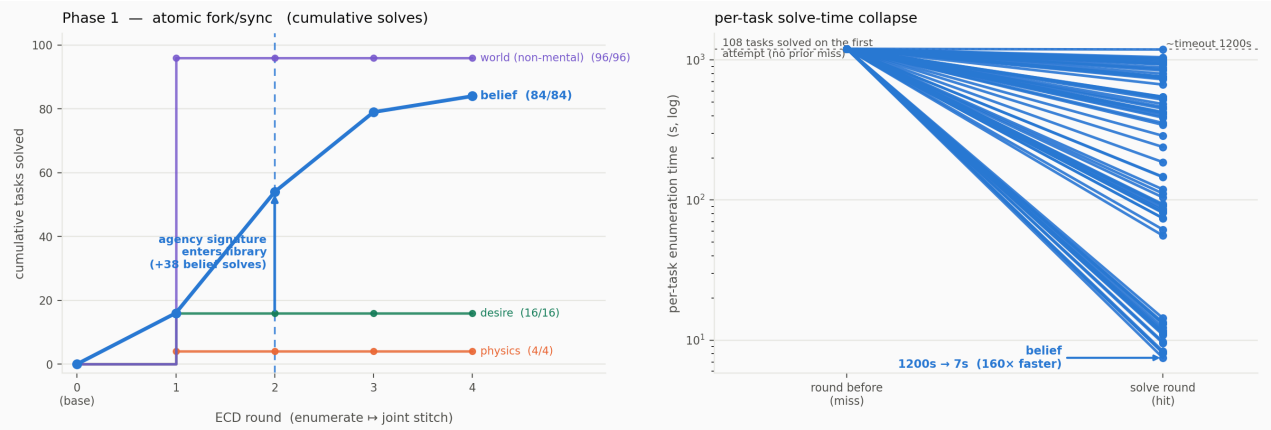


Figure 5: Cumulative solves by task family (atomic primitives)

Tasks other than belief get solved in the first round, they don't need abstractions. But belief can't be enumerated from base primitives, so it exhibits an S-curve. There are very few solves early on, since each solution must be laboriously composed of deep nested primitives. Then there's a sharp step up when the stitch finally abstracts the agency signature, the shared-model structure belief reuses. Then the learned abstraction is used to solve the remaining belief tasks, so the solved tasks plataeu for the last round.

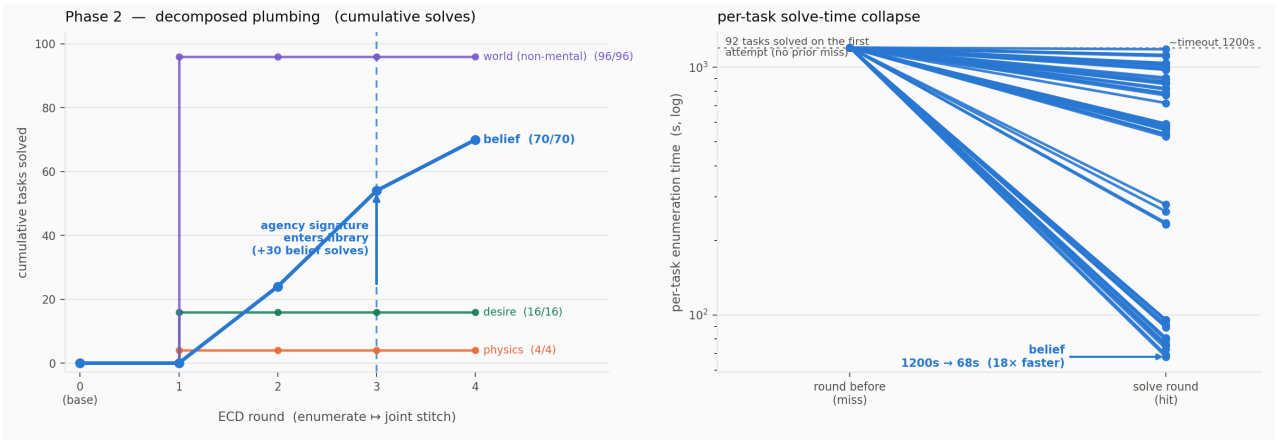


Figure 6: Cumulative solves by task family (decomposed primitives)

Starting with the atomic library, the abstraction jump occurs at round 2 where we abruptly see +38 belief solves. If we start with the decomposed library, the abstraction takes an extra round to assemble so the step is at round 3 where we see +30 belief solves.

Plot 2: per-task solve time, round n vs round n+1, as a paired slope-graph or log-scale scatter — the 1100 s → 10 s collapse is currently buried in log text.

5.4.1 Per-variant belief coverage + combo

Plot 1: Bar chart of solved/total for wall/witness/goal/dual

Plot 2: heatmap of (gv,av) × abstraction-used showing fn_8 tiling every cell.

Rows are the (goal value, agent value) combinations in a task, columns are the invented abstractions added to the library. The count indicates how many solutions use that abstraction.

INSERT FIGURE

So the belief abstraction generalizes across the (goal, agent) combinations it was trained on.

5.4.2 Behavioral probe: Passing the false-belief test

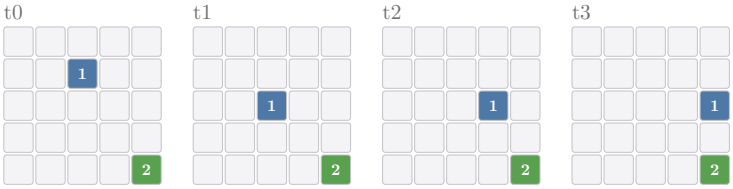
The analyses so far are about description length, showing that the intensional program is the shorter account of the data thanks to the abstraction that makes it easier to express more complex programs. We now show that the system passes the false belief test, reproducing the classic Sally-Anne false-belief setup [WP]. The test simply asks where the system predicts the agent will look. An object is where the agent last saw it, but the agent's belief about its location is now false; a child who attributes belief points to the believed location, while a younger child who cannot yet attribute belief points to the object's true location.

We use the goal-displacement family of tasks for this. These tasks depict a value's trajectory that changes direction 90°

Goal displacement — Sally-Anne [belief_goal]

```
(fork (seq (step gv d) (optimize (neg_dist gv) av)) (sync_to_world av))
```

av=1, gv=2, displaced_to=(2, 4), dirs=('up', 'up')



, because unlike the wall and witness families it lets the two readings come apart behaviorally. In a wall-belief scene the extensional rival posits a transient wall in the actual world rather than positing a wall in the agent's private believed world. That rival reproduces the entire trajectory, which is why the MDL margin, not behavior, is what separates them there. But in a goal-displacement scene the agent walks to where it believes the goal is (one cell from the true goal, which never moves) so a program that does not represent the agent's belief cannot reproduce the walk. The two readings predict the agent settling on different cells, which is the false-belief test.

The probe reuses the phase 1 run and re-stitches its solutions to rebuild exactly the library of abstractions it converged to (this is the same reconstruction that the MDL-margin experiment performs). Onto a held-out scene (drawn from a fresh seed that never entered the run, and checked to be absent from the training corpus) we instantiate the discovered belief compound (the `fork(derive, commit)` abstraction the library reprices belief through) and, as its foil, the shortest program expressible in the same library. Figure 7 shows one such scene.

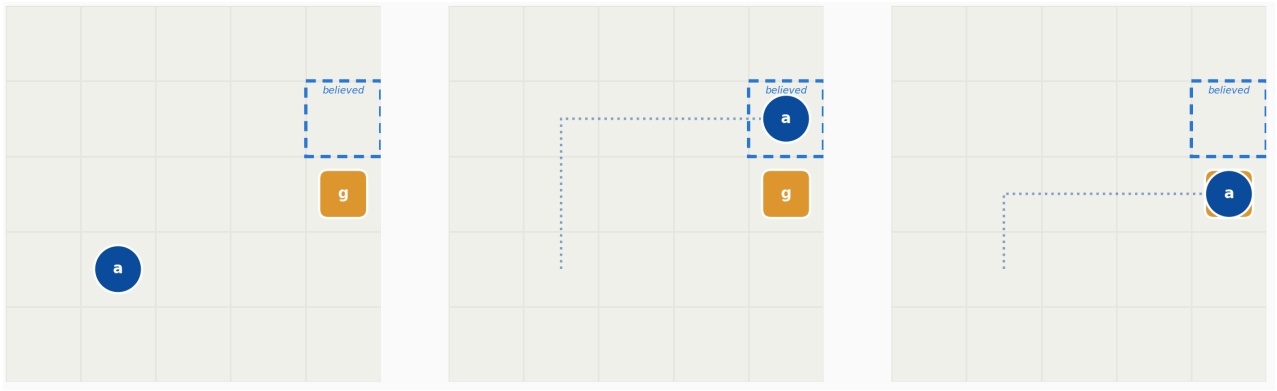


Figure 7: Behavioral probe on a held-out Sally-Anne scene. The learned belief compound sends the agent to the believed cell (passes); the shortest non-mental program under the same library sends it to the true goal (the naive answer).

With `fn_1` as the abstraction

```
(compose
  (optimize (neg_dist #1) #0)
  #2)
```

The learner equipped with the belief compound writes the program

```
(fork
  (fn_1 1 2 (step 2 down))
  (sync_to_world 1))
```

Which is normalized as

```
(fork
  (compose
    (optimize (neg_dist 2) 1)
     (step 2 down))
  (sync_to_world 1))
```

the discovered `fork(derive, commit)` abstraction applied to a derive that privately displaces the goal and seeks it, committing only the agent's own move back to the world via `sync_to_world`. Its final frame puts the agent on the believed cell, one step past the true goal. It searches where the agent thinks the goal is. The learner restricted to the non-mental fragment writes the shortest thing that fragment can say, plain desire `(optimize (neg_dist 2) 1)`, and its agent walks straight to the true goal, where the goal really is. Across all 24 held-out scenes the belief compound lands on the believed cell every time, the shortest non-mental program on the true cell every time.

Note that here the non-mental program is shorter than the intensional one (7.9 nats against 20.5), which is the opposite of the MDL-margin result, and this is the point. Where the transient-wall rival reproduces the scene, it does so only at a longer description length, so MDL rules it out. Where the pure-desire rival is cheaper, it does so only by predicting the wrong behavior, so the data rule it out. There is no non-mental program that is both as short and as accurate as the belief compound. The naive reading is available and parsimonious but it fails the test. A learner that has compressed its experience into a belief compound passes the false-belief task. a learner confined to the non-mental fragment fails by predicting the true location rather than the believed one. The abstraction the loop discovers for the sake of compression what licenses the mentalistic prediction.

5.5 Ablation experiments

5.5.1 Silo vs joint stitch

compress belief solutions alone vs pooled with the non-mental corpus. Is the constructor still found, and at what DL?

Plot: library-DL and constructor-invention bar pairs.

5.5.2 Corpus-mix dose-response

vary the non-mental fraction (0%, 25%, 50%, …); measure constructor invention and belief solve rate. Shows the non-belief tasks are the scaffolding that gets us to abstract belief

Plot: solve rate / invention frequency vs mix.

5.5.3 Budget dose-response

–t-fn at 600/1200/2400 s; belief solve rate per variant. Supports argument that misses are budget-bound, not due to representational limitation

Plot: solve rate vs budget, per variant (dual should be the steepest riser).

5.5.4 Dreaming ablation

–no-dream vs dreamed Q: cumulative solves vs wall-clock. Round 3-vs-4 in the log already hints the dreamed Q matters.

plot a curve that shows Q bias contribution

5.5.5 curriculum ablation

remove certain tasks, show that low-level abstractions don't get abstracted and then that prevents us from reaching high level abstractions

5.5.6 Seed robustness

5 corpus seeds × the main run; report every verdict metric as k/5 with solve-rate error bars

5.6 Discussion and rebuttals

question: to what extent is the ToM already present in the input data and DSL and structure

potential argument: `fork` and `sync_to_world` are just decomposed `believes`. By your definition you'be we've been using from the literature, ToM/mentalizing is to be able to attribute a belief to an agent. A belief is an intensional state, a grid representation that may differ from the world. So `fork(derive, commit)` is already belief attribution, you've just broken it apart into components and now they're put back together.

Responses:

1. `fork` and `sync` have non-mental uses, and they're often used independently of each other to solve other things, so they're general-purpose
2. there are also lots of other primitives that are the duals (cube axis), and those don't get selected

cube of symmetric complements, and general pair interface (`overlay`, `registration`) that stops `fork`/`sync` from being a disguised `believe`.

potential argument: the `fork` primitive smuggles in ToM because creating a copy of the world grid is a representational capacity

response: ToM isn't just the representational capacity. creating a copy of the world that differs from it is just counterfactual reasoning, the what-if ability. it's super domain-general. infants have been shown to have it, disjunctive reasoning. The ToM is an abstraction that *uses* this domain-general counterfactual representaion capacity to attribute a counterfactual world to a particular agent - that this modified world is *that agent's* world.

potential argument: `optimize` is basically ToM because BFS navigation requires the agent to recognize a

out of the full 2×2×… cube of channel-direction / channel-target / z-order / scope variants, joint MDL still selects exactly the read-model→write-world, single-av corner for belief, and leaves the symmetric variants attached to their non-mental tasks.

pair apparatus was available to every task, but recruited by only the false belief ones. Those are the trajectories that can't be accounted for by only a single grid, so the search pays + 10 nats for

the second grid only when nothing cheaper explains the data. Representationalist's criterion: posit the hidden world only if it pays for itself, only if its necessary. Availability is not use. The inference "this agent needs a belief" is made, per task, by cost-driven model selection.

The `(grid,grid)` and arrow types aren't smuggled-in ToM, they're just the logical form that ToM requires. Intensionality is the coexistence of world and model. Any substrate expressive enough to even state "act on the world via a transformed model of it" must be able to hold two representations at once. And "two things held at once" is a product, in some guise (a pair, two registers, two tape regions, two variables). You cannot discover ToM in a substrate that can't hold two representations [for the same reason you can't discover addition in a language with no notion of "two numbers." The pair is to theory of mind what the integers are to arithmetic.] And the pair is symmetric and content-free: `dup` makes `(w, w)`. There's nothing in it that says "the second one is the model, the believed world." The asymmetry that makes one copy real and the other believed is: transform the second (`mapsnd`, not `mapfst`), commit from model to world (`sync_to_world`). This asymmetry is entirely in the composition, which is the discovered part. The medium is neutral, it could go any way, as seen in the other tasks which do use those other primitives and wire them in other ways. But the system is able to discover this particular composition of these particular primitives to solve belief tasks, via programs that constitute a theory of mind.

Sure you can keep going. Decompose `dup` from a flat memory/tape with allocate/read/write, so "hold a second world-model" becomes a discovered pattern of buffer use. But be clear-eyed: that relocates the regress, it doesn't end it. "Allocate a buffer" is then the primitive, and it's no more or less mental than "form a pair" — the capacity-for-plurality is presupposed by intensionality at every level, because it's constitutive of intensionality. There is no substrate that discovers ToM "from nothing"; there's only substrates whose primitives are, or aren't, individually non-mental and general.

generality by reuse is what shows it

1. the same combinators used to create the belief abstraction are used in non-mental tasks
2. the library is also initialized with symmetric primitives to the ones used in the belief abstraction

so it's a domain-general library that the search happened to recruit for belief.

Chapter 6

Further work

6.1 Comparison to LLMs

I'm doing search in program-dynamics space, not token space. Synthesizing programs is logical and semantic discovery, unlike token prediction. ECD enumerates typed program trees by probability and runs them on grids; there is no surface-syntax statistics anywhere.

6.2 Polymorphism

We use a simply-typed monomorphic type system. For our purporses, types are really used only to prune enumeration.They constrain which compositions are legal, not what the program must prove. DreamCoder uses polymorphic types, so unification can rule out ill-typed branches early. But since our DSL has only ∼30 primitives our bottleneck isn't branching factor, so monomorphic dict lookup is sufficient.

With monomorphic types, a stitch abstraction discovered at type `grid` can only be reused at `grid`. With polymorphism, an abstraction abstracted over $\forall a$ can be reused across instantiations of any type a. Phase 3 hand-engineers this with t eh cons/nil stack leaving arity as a free parameter. If we had polymorphism it would just fall out of the type discipline.

To do something like that, we'd replace bare string types with a small ADT:

- ground `('con', 'grid', [])`
- variable `('var', 'a')`
- applied `('con', 'pair', [t1, t2])`
- arrows `(args, ret)`

We'd implement Robinson unification

6.3 persistent models and interaction

currently an attributed grid state, a belief, exists only for that one frame, since we're just working with grid to grid transition functions. so do something where the agent actually has a persistent memory of its modified grid

and something where the agent can actually react to its environment, can write to the private model and read from it, can navigate based on obstacles, etc

6.4 breaking down optimize

this primitive packs in a lot, it's a full bfs search. so try to rediscover that. this requires a persistent model (to keep track of visited cells) and a and interactions with the environemtn, conditionals (if wall, move away)

6.5 probabilistic primitives

an agent that sometimes does this and sometimes that, etc

6.6 vs. real world

obviously this isn't how an infant learns theory of mind

I'm missing the *self*. I'm focusing on learning to distinguish between different entities in the domain, but not an infant learning to distinguish others from itself. For example, if I bite myself it hurts, but if I bite someone else it doesn't. maybe pain occurs only if I bite this but not that (extensional). or we have different states, different experiences, and sometimes someone else can be experiencing pain even when I'm not.

I'm missing the scale. I'm focusing on a narrow timescale. in real infants this would unfold over a long time scale.

6.7 richer belief content

6.8 recursive mindreading

Chapter 7.

Appendix

This appendix collects the implementation of the model outlined in Chapter 3.

- full list of tasks
- full list of primitives

7.1. The interpreter

The interpreter is discussed at the model level in Section 7.1. What follows is the design argument in the form it was originally worked out, followed by the two interpreters themselves.

Say we have a candidate program ρ, so the enumerator needs to check for each task x in the corpus whether ρ solves task x. A program ρ solves task x iff running ρ on initial frame x_0 outputs the entire history x. So every solution should be a program that takes as input a $d \times d$ frame and returns a $d \times d \times n$ stack of frames, a program of the form: "iteratively apply transition function f, starting with initial frame x_0, for n timesteps". So every solution program has the same outer structure, with only f varying between programs.

This presents a problem: this shared structure, present in every program, will be baked-in to every abstraction. For example say ten tasks in a corpus depict some object falling down, so stitch should identify `(step down #0)` as a shared structure and add this "gravity" primitive to the library. But because "apply f, starting with x_0, for n timesteps" is a structure common to all programs, stitch would add an overspecified abstraction $f_1 := \lambda$ `0 1`. *iteratively apply* `(step down #0)`*, starting with* x_0*, for* `#1` *timesteps* rather than the more general $f_2 := \lambda$ `0`. `(step down #0)`.

The problem is that f_1 doesn't compose, which defeats the purpose of library learning. Say we abstracted f_1 as "gravity", and we now want to use it to solve a task involving two objects both dropping to the ground. We can't do that, because f_1's output is of type `matrix`, an entire 3d scene depicting just one falling object. But if we've got an f_2 abstraction with output type `fn`, we can create a program like `(compose (fn_2 4) (fn_2 5))` that reuses the abstraction for each component of a scene within that same scene (where `compose` is of type `fn -> fn -> fn`).

Since "iteratively apply transition function f, starting with initial frame x_0, for n timesteps" is common to every program, what the enumerator is looking for is really just that transition function f. The rest is just what's needed to render a scene given a transition function, to check whether f correctly produces a given task scene. So we simply move the rendering machinery from program space to the evaluation framework.

We implement an interpreter that, given a candidate program (transition function) f, a task's initial frame x_0, and the task's duration (z-dimension) n, generates a scene x' by applying f iteratively (starting with x_0) for n timesteps.

```
def unfold(g: grid, T: int, f: fn) -> mat:
    frames = [g.copy()]
    for _ in range(T - 1):
        g = f(g)
        frames.append(g.copy())
    return np.stack(frames)
```

`unfold` is the anamorphism that at each timestep applies f to the previous grid to generate the new grid, and appends the new grid to the list of grids. So `unfold`'s state is just a single grid: it doesn't encode a world/model grid pair, nor a registry of agents and goals. A transition function f is a `grid` endomorphism `fn :: grid -> grid`.

For some of the tasks, we use an interpreter for programs of root `fn_p_g :: (grid, grid) -> grid` since those tasks don't involve just one input grid but a pair. That uses `unfold_with_template(g, template, T, c)`

```
def unfold_with_template(g, template, T, c):
    """grid, grid, int, fn_p_g -> mat: thread g; each frame pair it with a
    *constant external* template and apply commit c.  Unlike fork, the second
    channel is a given input, not a privately derived model — so a program that
    uses sync_to_world here is doing registration, not holding a belief.
    """
    frames = [g.copy()]
    for _ in range(T - 1):
        g = c((g, template))
        frames.append(g.copy())
    return np.stack(frames)
```

7.2. Program representation

The hypothesis space is described at the model level in !types. This is the data structure that represents it.

Every program, primitive, and invented abstraction is a `Delta`, i.e. an object that can serve as a node in an expression tree. A `Delta` carries a `head` (the Python value or callable it denotes), a `type` (the type its subtree returns), a list of `tailtypes` (the types of the arguments it expects), and a list of `tails` (the actual argument subtrees, once they are filled in). So a `Delta` with no `tailtypes` is a *terminal*, a leaf whose value is just its head, for example `up :: dir` or the literal `4 :: cellvalue`. A `Delta` with `tailtypes` is an *operator*: an interior node like `step :: cellvalue, dir -> fn`, whose head is applied to its filled-in tails. Then a node's `arrow` is the signature that the enumerator matches against when it fills argument slots. For a terminal, its arrow is just its `type`. For an operator, it's the pair `(tailtypes, type)`, the arrow from its input types to its output types.

The expression tree is directly executable. Calling a `Delta` evaluates the program it represents, so a terminal returns its head while an operator recursively evaluates each of its tails and applies its head to the results. So the program `(step 4 up)` is a `step`-headed node with tails `4` and `up`; calling

it evaluates the two leaves to the cell value `4` and the direction `(-1,0)`, then applies `step` to obtain a transition function `fn :: grid -> grid`.

An invented abstraction is represented as a `Delta` as well, but it's distinguished by a `hiddentail` which is the body of the abstraction. The `hiddentail` is itself a `Delta` tree over lower-level primitives, with numbered argument holes `$0, $1, ...` standing in for the abstraction's parameters. When such a node is called, its body is copied and each hole `$i` is replaced by the corresponding tail before the body is evaluated. This is what makes an abstraction cost a single token. The whole subtree it abbreviates is hidden inside the `hiddentail` and does not appear in the program that uses it.

The description length of a program is its node count, computed by `length`. a leaf counts one, and an operator counts one plus the lengths of its tails. So an invented abstraction used as a token adds just one, since its body lives in the un-charged `hiddentail`. This node count is the minimum-description-length quantity the enumerator uses to tiebreak between programs that solve a task within the same budget window (see !enumeration).

7.2.1. The library object

The library of primitives is a `Deltas` object. It is instantiated with a list of `core` primitives and grows a parallel list of `invented` abstractions as compression discovers them. When the enumerator draws primitives, it draws from the concatenation of `core` and `invented` which is the full library. Whenever a primitive is added, `Deltas.infer` rebuilds the indices the search depends on — most importantly `bytype`, which groups the alphabet's symbols by their return type. This grouping is what the type-uniform prior is computed from: a symbol of type τ is priced at $-\ln$ of the number of symbols sharing type τ, so `bytype` gives both the branching factor at each argument slot and the cost of each choice (!enumeration).

7.3. Task generators and necessity filters

The identifiability condition on the mental families is stated in !identifiability. It is enforced by the generators as follows.

one of the challenges of creating such a corpus is making tasks that require belief attribution.

checks in the (belief) task generators to make sure they aren't physically explainable, so they're not underdetermined. For example, in a wall false belief task an agent might have a false belief about a wall being there, but at a location nowhere near the agent's path to its goal. So the agent can still navigate to the goal freely as if it didn't have such a false belief, positing the false belief isn't necessary or helpful to explain the scene. So those scenes are rejected.

we implement a filter that

7.4. Cost-bounded enumeration

The search is described at the model level in !enumeration. These are its mechanics.

While a single cost-ordered stream can in principle be checked against every task in the corpus at once, in the experiments each still-unsolved task is instead enumerated as its own cost-ordered stream, parallelized across tasks. This per-task granularity is what lets the prior Q be conditioned

on the individual scene rather than shared across the corpus — a fact we rely on in !dreaming, where Q becomes a task-conditional distribution $Q(\cdot \mid x)$ emitted by a recognition model.

Program search proceeds by budget window to ensure that cheaper programs are enumerated first. For `(step 4 up)`, the first two windows are skipped by pruning. We start enumerating in window $[0, 2)$, but since `step`'s cost is 2.1 nats it exceeds the budget, so it cannot be chosen as the root node of the program tree and we proceed to budget window $[2, 4)$. After paying 2.1 nats for `step`, only 1.9 nats remain to fill the two holes in the partial program tree. `step`'s tail types (arguments) are `cellvalue` and `step`. But any primitive of type `cellvalue` costs 2.3 nats, so `step`'s first argument can't be placed. So trees with a `step` node are pruned, since they would never complete within the second budget window. Search is cheapest-first, since more complex programs only become affordable once the window is wide enough.

At budget window $[4, 6)$ the ceiling is finally sufficiently wide to admit all three symbols. The tree is assembled incrementally: each enumerated subtree is appended to a copy of its parent node, and a callback fills the remaining argument slots by the same recursion (so programs of any arity or nesting depth are handled by the same procedure). Since enumeration is type-directed, each argument slot draws candidates only from symbols of the matching type, so no ill-typed partial tree is ever constructed. `(step 4 up)` completes at its total cost of ≈ 5.8 nats.

When `(step 4 up)` is found the tree rooted by `step` has no more available holes, so the callback fires to evaluate the found program ρ. For each task x in the corpus, the interpreter generates the output matrix given the input grid of the task by calling `unfold` on ρ, initial grid x_0, and x's third dimension n. Then the resulting output matrix x' is hashed and compared against task x's hash. So for a program to solve a task, it must produce a grid that exactly matches that task's entire trajectory, not just the endpoint of the trajectory. This ensures that programs capture the full behavior rather than just the end result.

If `(step 4 up)` is the first solution found for the task, it's stored as the task's solution. Because the windows are tried in increasing order of cost, the first solution the search returns is in principle the shortest program that solves the task. But enumeration is parallelized so search unfolds simultaneously over program trees, so we need to tiebreak when multiple programs solve a task within the same budget window. To tiebreak we first run `Delta.simplify` on the programs, a semantics-preserving rewrite that collapses spurious nesting which sometimes occurs during enumeration. We then run `Delta.length` to get the node count of the expression trees we need to compare. Every operator and leaf adds to the count, and an invented abstraction used as a token adds one to the count too. The body of the abstraction isn't charged, since the components now count for just application. The program with the lower node count wins since it has the lower minimum description length (since by symbol count it's shorter).

Also, the search space contains lots of junk programs that crash or return nothing even though they type-check. So the enumerator callback wraps both the interpreter and the evaluator in `try/except` blocks and skips any degenerate or non-callable program.

7.5. Compression with Stitch

The compression objective and the tradeoff it makes are given in !compression. This is the search that optimizes it.

In DreamCoder [Ell+20] the compression step is bottom-up: for each found program it enumerates the exponential set of program refactorings (stored as version spaces) and intersects them to grow abstractions up from reoccurring subexpressions. Rather than this bottom-up approach, we delegate the search for abstractions to the top-down Stitch [Bow+23] library-learning system. Instead of enumerating refactorings of every program, stitch searches the space of abstractions directly. So stitch doesn't have to materialize every rewrite.

To create an abstraction, stitch starts with a tree that consists of just a root node `??`, an unexpanded hole that the search still needs to refine. At the root `??`, the abstraction *matches* every subexpression, since it's the most general abstraction possible: any subexpression can be trivially "rewritten"

. As the tree is built up and the search is refined, the set of possible match locations shrinks.

A node of the stitch abstraction search tree is a partial abstraction. It consists of some combination of primitive operations (like `step`) and argument holes `#i`, and at least one unexpanded hole `??` that the search still needs to refine.

At each step it commits part of the abstraction's body to a concrete operator or splits a subtree, branching downward toward more specific abstractions.

upper bound for a partial abstraction $A_{??}$ [Bow+23]

$$U_{\text{upperbound}}(A_{??}) = \sum_{e \in \text{ matches}(A_{??})} \text{size}(e)$$

goal of compression is to minimize the size (cost) of program corpus after rewrite. Compressive utility function:

Each compression phase consists of a fixed number of stitch iterations, determined by the `iterations` parameter. Each iteration searches for the single most compressive abstraction, rewrites the corpus in terms of it, and repeats, so later abstractions may be built out of earlier ones. The `max_arity` parameter bounds how many holes (`#0`, `#1`, …) an abstraction may carry.

7.5.1. Normalization before compression

Before passing the found solutions to stitch for abstraction, we first need to fully expand any existing primitives in those solutions. For this we implement `normalize`, the complementary operation to abstraction. It fully inlines every invented `fn_k` back to the bare primitive alphabet, recursively substituting each abstraction's argument holes with its actual arguments until nothing but core primitives remain.

We need to normalize because stitch's top-down compression can only abstract over structure it can see, and an opaque `fn_k` token has no visible interior. So any abstraction that would need to reach inside `fn_k`, or span its boundary and surrounding context, is simply invisble to the search.

the compression step must be handed programs in a common vocabulary — feeding it already-compressed programs, in which a prior round's fn_0 appears as an opaque token, would let those tokens collide with the fresh abstractions the compressor invents under the same names. Normalizing before compression guarantees the compressor sees only primitives and the structure it discovers is genuinely its own. Renaming it to `inv_0` avoids the lexical clash but leaves it just as opaque; stitch still can't see the `(compose (wall_at …) (optimize …))` sitting inside it. Inlining flattens the token back to that primitive structure, and only then can the compressor find abstractions that cut across the old boundaries.

The best syntactic fragment to abstract from round N's solution may carve the primitives differently than round $N + 1$'s. For example,

So we can refactor abstractions that were previously ideal but are no longer ideal

7.6. The recognition model

Amortization is described at the model level in !dreaming. This is the network, its training data, and the rescaling that puts it on the enumerator's cost scale.

In the first enumeration run, primitives are chosen under type-uniform prior distribution Q.

The recognition model `MatRecognitionModel` is built around a symbolic grid encoder rather than a generic convolutional one. Its central design commitment is that entity *roles are read off from motion, not from cell values*: because the corpus deliberately uses diverse agent and goal identifiers, the encoder cannot key on "value 1 is the agent." Instead it labels a cell occupied in frame x_0 but vacated by the final frame a *mover*, and a non-background cell occupied at both ends a stationary *goal*. It then pools row and column position embeddings into per-entity slots: two mover slots (each carrying both a start-position pool and a trajectory pool), two goal slots, a wall pool, and a path-length embedding of the duration T, concatenated into a single matrix embedding h_{matrix}. The per-entity layout is what lets the two-mover, two-goal scenes of the witness and dual belief families be represented in separate slots rather than blurred into one mean, and the separate per-mover trajectory pool is the decisive signal for false belief: a detour path and a straight path yield different mean agent positions even when their start, goal, and T coincide, which is exactly the case a belief task presents. A linear head maps h_{matrix} to logits over the whole library D.

Crucially the prediction is *flat*: one forward pass per task yields a single distribution over the DSL, computed once and reused for every enumeration decision on that task. We deliberately forgo a tree-context recurrent network that would condition each node's prediction on its partial parent tree. A per-node network is both far more expensive — it turns each of the thousands of enumeration steps into its own forward pass — and prone to failing exactly where it matters: the abstractions belief requires are deep, and a tree-conditioned model trained on shallow solved programs generalizes poorly to the depths it has not yet seen. A flat, matrix-conditioned Q sidesteps both problems.

Replays and fantasies. The model is trained by the wake–sleep procedure of a Helmholtz machine, on two sources of program–scene pairs. *Replays* are the actual solutions enumeration has found so far, run back through the interpreter on their own task grids — supervised examples of "this scene was solved by this program." But replays alone are scarce and biased toward the families already solved, so they are supplemented with *fantasies*: programs sampled directly from the current

library prior and executed by `unfold` on freshly generated grids to synthesize new (scene, program) pairs at no labelling cost. Each dreaming iteration draws up to four replays and fills the rest of a batch of eight with fantasies. The fantasy grids are not arbitrary — their size, entity values, and the fraction carrying four or more entities are all read off the corpus's own scene statistics, so the encoder trains on the scene mix it will face at test time (in particular the multi-entity scenes that exercise its second mover and goal slots). Roles remain decided by the sampled program's motion, so a fantasy only ever *lets* a program drive two movers; it never manufactures a belief scene by hand. For every pair the model encodes the resulting matrix and is trained by cross-entropy to predict each node of the generating program tree, averaged over nodes.

Putting the model on the enumeration cost scale. A recognition model trained this way emits a globally-normalized distribution over the library, which is on the wrong scale for the budget-window enumerator and, having been trained only on the families solved so far, could actively suppress the rare primitives an as-yet-unsolved family needs. `dreamed_q` reconciles the two so that dreaming can only ever help. First it re-softmaxes the model's logits *within each type group*, so a program's summed cost is comparable to the uniform and content priors rather than living on the model's global scale — otherwise a ten-node program would land many budget windows too late and time out before it is ever reached. Second it *floors the result at the uniform prior*, $Q = \max(Q_{\text{model}}, Q_{\text{uniform}})$: the model may make a primitive cheaper, and so enumerated earlier, but never push one below its uniform reachability, so every program reachable under the baseline stays reachable. Third it preserves the *content-literal boost* the baseline depends on, forcing any integer literal already visible in frame x_0 to cost zero. The model's confident guesses are explored first, while nothing it has not seen is priced out of reach.

Feeding back into enumeration. The trained model does not steer every family. Dreamed Q is applied only to families with at least one solved instance — a scene the recognition model was actually trained on as a replay. A zero-replay family, such as belief before its very first solve, is invisible to the model and could only be mispriced by it: it would flood the early budget windows with the non-belief primitives it *has* seen, each pushed below uniform, and thereby delay that family's own primitives past the timeout. Such families stay on the proven uniform and content baseline and earn the dreamed prior only once they are solved and can contribute replays of their own. In this way the recognition model accelerates the families the curriculum has already reached without ever slowing down the frontier it has not — and its benefit compounds with compression, since each round it is retrained over a library whose new abstractions let it steer toward deeper structure than the round before.

Bibliography

[MSS12] Margolis E, Samuels R, Stich SP, editors. The Oxford handbook of philosophy of cognitive science. Oxford New York: Oxford University Press; 2012. https://doi.org/10.1093/oxfordhb/9780195309799.001.0001.

[WP] Wimmer H, Perner J. Beliefsabout beliefs: Representationand constraircing function of wrong bekfs in young children'sunderstanding of deception n.d.

[FP] Fodor JA, Pylyshyn ZW. Connectionism and cognitive architecture: A critical analysis n.d.

[WL04] Wellman HM, Liu D. Scaling of Theory-of-Mind Tasks. Child Development 2004;75:523–41. https://doi.org/10.1111/j.1467-8624.2004.00691.x.

[WCW01] Wellman HM, Cross D, Watson J. Meta-Analysis of Theory-of-Mind Development: The Truth about False Belief. Child Development 2001;72:655–84. https://doi.org/10.1111/1467-8624.00304.

[GCT24] Griffiths TL, Chater N, Tenenbaum J. Bayesian models of cognition: reverse engineering the mind. Cambridge, Massachusetts: The MIT Press; 2024.

[Les94] Leslie AM. Pretending and believing: issues in the theory of ToMM. Cognition 1994;50:211–38. https://doi.org/10.1016/0010-0277(94)90029-9.

[Les05] Leslie AM. Developmental parallels in understanding minds and bodies. Trends in Cognitive Sciences 2005;9:459–62. https://doi.org/10.1016/j.tics.2005.08.002.

[SL01] Scholl BJ, Leslie AM. Minds, Modules, and Meta-Analysis. Child Development 2001;72:696–701. https://doi.org/10.1111/1467-8624.00308.

[Ten+11] Tenenbaum JB, Kemp C, Griffiths TL, Goodman ND. How to Grow a Mind: Statistics, Structure, and Abstraction. Science 2011;331:1279–85. https://doi.org/10.1126/science.1192788.

[Car11] Carey S. The Origin of Concepts. Cary: Oxford University Press USA - OSO; 2011.

[PTG12] Piantadosi ST, Tenenbaum JB, Goodman ND. Bootstrapping in a language of thought: A formal model of numerical concept learning. Cognition 2012;123:199–217. https://doi.org/10.1016/j.cognition.2011.11.005.

[Mar10] Marr D. Vision: A Computational Investigation into the Human Representation and Processing of Visual Information. The MIT Press; 2010. https://doi.org/10.7551/mitpress/9780262514620.001.0001.

[PTR11] Perfors A, Tenenbaum JB, Regier T. The learnability of abstract syntactic principles. Cognition 2011;118:306–38. https://doi.org/10.1016/j.cognition.2010.11.001.

[Rul] Rule JS. Pre´cis of The child as hacker: Building more human-like models of learning n.d.

[Bru+09] Bruner JS, Goodnow JJ, Austin GA, Brown R. A study of thinking. 6. printing. New Brunswick: Transaction Publishers; 2009.

[KPT07] Kemp C, Perfors A, Tenenbaum JB. Learning overhypotheses with hierarchical Bayesian models. Developmental Science 2007;10:307–21. https://doi.org/10.1111/j.1467-7687.2007.00585.x.

[KT08] Kemp C, Tenenbaum JB. The discovery of structural form. Proceedings of the National Academy of Sciences 2008;105:10687–92. https://doi.org/10.1073/pnas.0802631105.

[GUT11] Goodman ND, Ullman TD, Tenenbaum JB. Learning a theory of causality. Psychological Review 2011;118:110–9. https://doi.org/10.1037/a0021336.

[Lak+16] Lake BM, Ullman TD, Tenenbaum JB, Gershman SJ. Building Machines That Learn and Think Like People 2016. https://doi.org/10.48550/arXiv.1604.00289.

[BST09] Baker CL, Saxe R, Tenenbaum JB. Action understanding as inverse planning. Cognition 2009;113:329–49. https://doi.org/10.1016/j.cognition.2009.07.005.

[Ell+20] Ellis K, Wong C, Nye M, Sable-Meyer M, Cary L, Morales L, et al. DreamCoder: Growing generalizable, interpretable knowledge with wake-sleep Bayesian program learning 2020. https://doi.org/10.48550/arXiv.2006.08381.

[Bow+23] Bowers M, Olausson TX, Wong L, Grand G, Tenenbaum JB, Ellis K, et al. Top-Down Synthesis for Library Learning. Proceedings of the ACM on Programming Languages 2023;7:1182–213. https://doi.org/10.1145/3571234.